



UT5: TRATAMIENTO DE DATOS

Néstor Sabater Hernández

1. INTRODUCCIÓN

- En esta unidad nos centraremos en la **modificación de la información almacenada en una base de datos** existente:
 - Utilizando tanto herramientas gráficas proporcionadas por el gestor, como herramientas externas al mismo.
 - Realizando esas modificaciones con las sentencias SQL que forman parte del lenguaje de manipulación de datos (LMD), en concreto con las sentencias **INSERT, UPDATE Y DELETE**.
 - Además, para mantener la integridad y consistencia de la información, **aprenderás a agrupar estas sentencias dentro de una TRANSACCIÓN**.
-

1. INTRODUCCIÓN

- LMD
 - A las consultas que utilizan la sentencia **SELECT** se les denomina "**Consultas de selección**".
 - A las consultas que permiten la edición de los datos con **INSERT**, **UPDATE** ó **DELETE** se les denomina "**Consultas de actualización**" o "**Consultas de acción**".

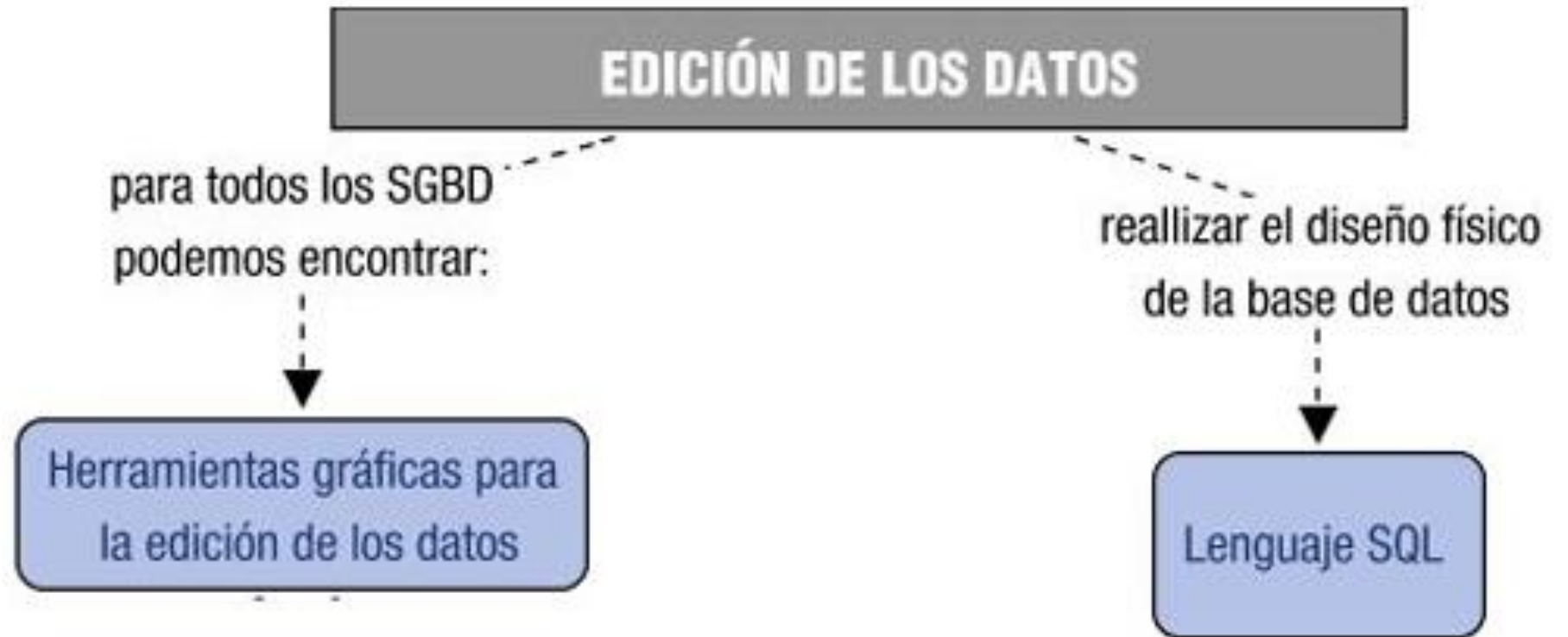
1. INTRODUCCIÓN

- Consultas de edición de datos - LMD (SQL)



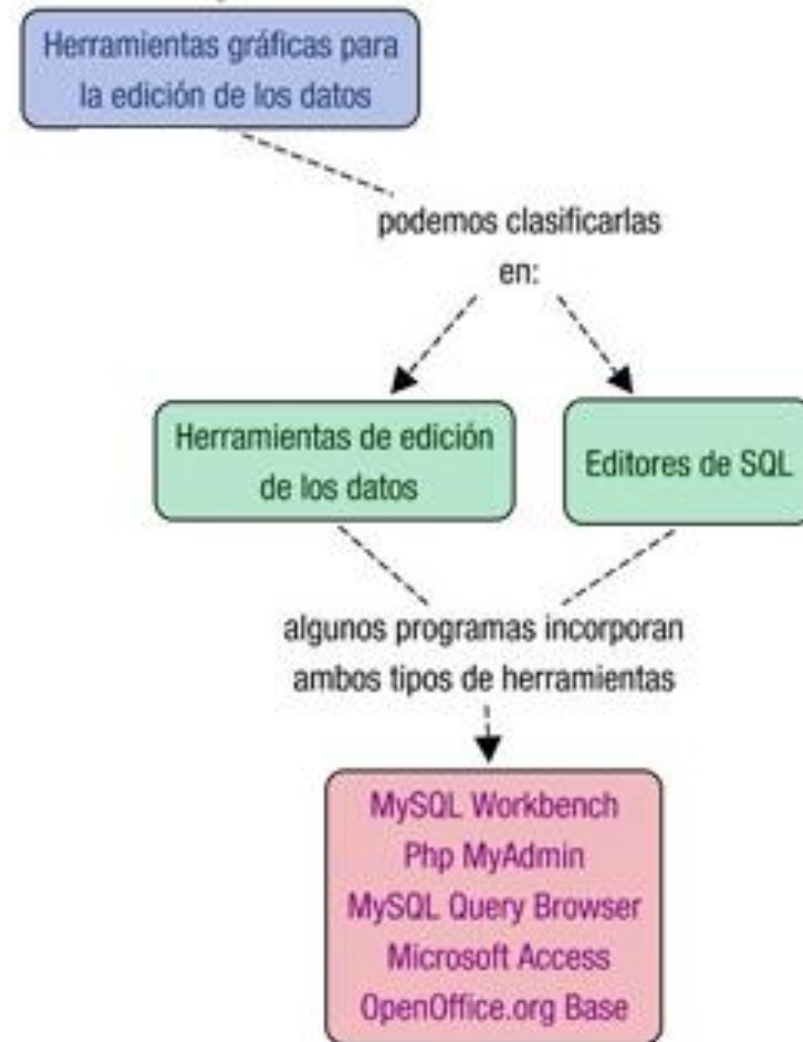
1. INTRODUCCIÓN

- Edición de Datos



1. INTRODUCCIÓN

- Edición de Datos



2. HERRAMIENTAS GRÁFICAS

- Como venimos tratando en unidades anteriores, los **SGBD disponibles en el mercado incorporan herramientas gráficas** para que tareas como el diseño, la creación de bases de datos y de las tablas y la consulta de la información almacenada sea más sencilla.
 - Lo mismo ocurre con la edición de los datos, tanto dentro del SGBD como mediante aplicaciones externas al propio gestor podemos añadir, modificar o eliminar filas de las bases de datos desde una ventana de edición gráfica sin tener que escribir código SQL.
-

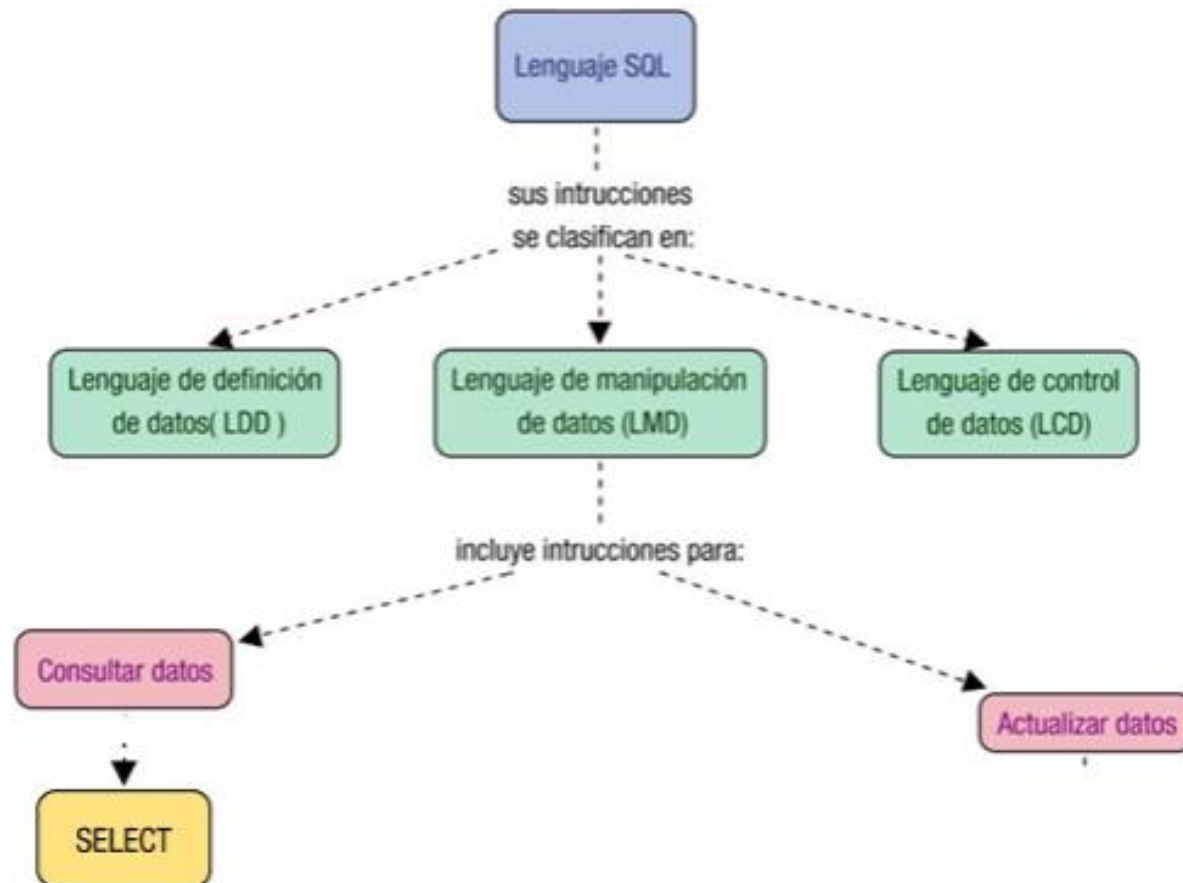
2. HERRAMIENTAS GRÁFICAS

- **Estas herramientas facilitan las tareas cuando pretendemos realizar acciones sencillas, tareas rutinarias o cuando el usuario no tiene conocimientos de programación**, pero a lo largo de las unidades vistas hasta el momento ya hemos podido llegar a la conclusión de que cuando se trata de acciones complejas en las que necesitamos tener un mayor control de los datos que se van a alterar será necesario recurrir a nuestros conocimientos de SQL.
 - Algunos editores gráficos que permiten actualizar las filas de una tabla:
 - PhpMyAdmin
 - MySQL Workbench
 - Open Office Base
 - Microsoft Access.
-

3. CONSULTAS DE EDICIÓN DE DATOS – LMD (SQL)

- Las consultas que vamos a tratar en esta unidad corresponden al lenguaje de manipulación de datos LMD (al igual que la sentencia SELECT)
- La diferencia es que hasta ahora hemos visto consultas que devuelven datos.
- **Las consultas que vamos a ver a continuación cambian los datos almacenados en las tablas.**
 - Nos referimos a las consultas **INSERT, UPDATE y DELETE.**

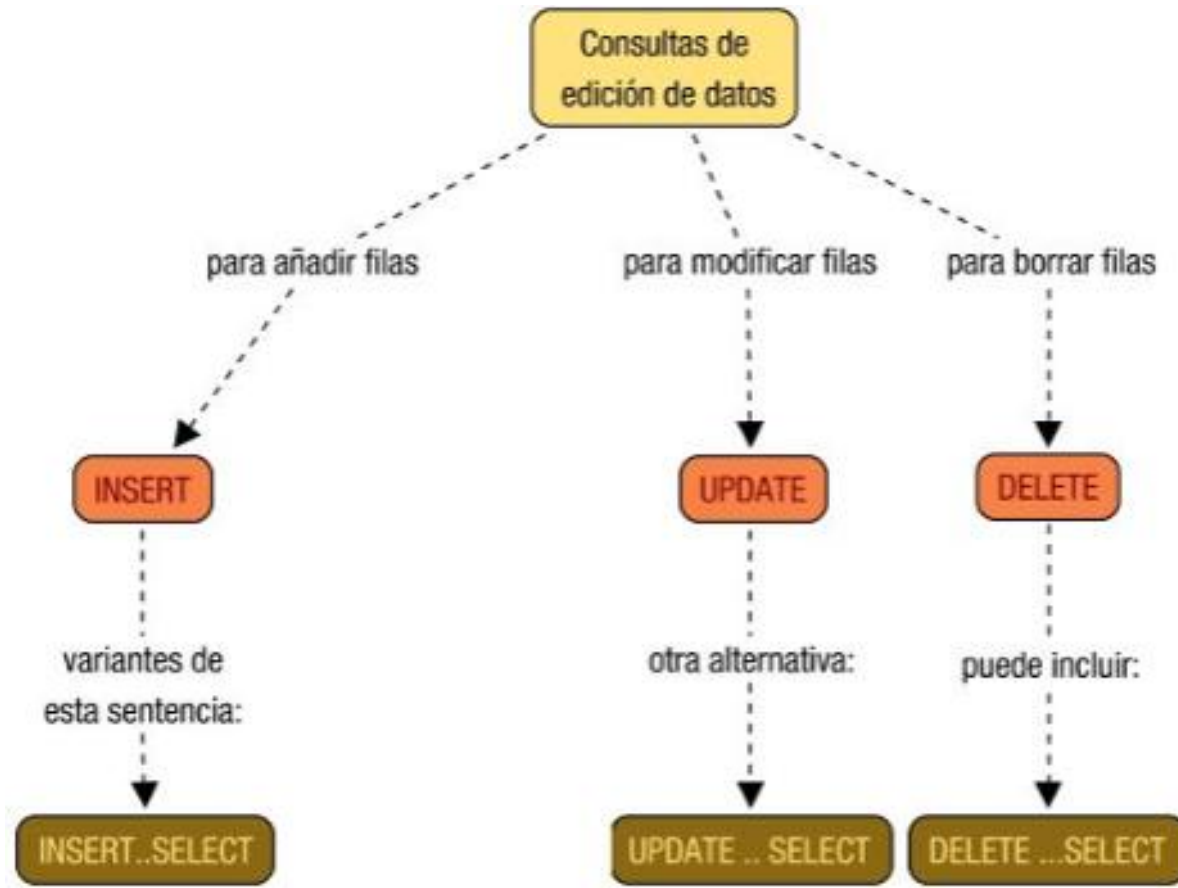
3. CONSULTAS DE EDICIÓN DE DATOS – LMD (SQL)



3. CONSULTAS DE EDICIÓN DE DATOS – LMD (SQL)



3. CONSULTAS DE EDICIÓN DE DATOS – LMD (SQL)



4. INSERCIÓN DE DATOS

- La sentencia INSERT de SQL
 - Permite insertar una fila en una tabla
 - Es decir, añadir un registro de información a una tabla.
- Es posible introducir los valores directamente en la sentencia o introducirlos a partir de la información existente en la BD mediante un SELECT.

4. INSERCIÓN DE DATOS

- Sintaxis
 - **INSERT INTO** nombretabla [(nombre_columna,)] **VALUES** ({valor | DEFAULT});
 - nombretabla es el nombre de la tabla donde se quiere insertar la fila.
 - [(nombre_columna,)] representa la columna o columnas donde se van a introducir valores.
 - Especificar las columnas es opcional
 - **Si no se especifica ninguna columna, se consideran por defecto todas las columnas en el mismo orden que la definición de la tabla.**
-

4. INSERCIÓN DE DATOS

- Sintaxis
 - **INSERT INTO** nombretabla [(nombre_columna,)] **VALUES** ({valor | DEFAULT});
 - {valor | DEFAULT} representa los valores que se van a asignar a las columnas.
 - **Si no se especifican las columnas:**
 - Se han de introducir valores en todas las columnas
 - La lista de valores se escribirá conforme al orden de las columnas en la definición de la tabla.
 - **Se deben corresponder los tipos de datos.**
-

4. INSERCIÓN DE DATOS

- EJEMPLO

```
mysql> desc empleados;
```

Field	Type	Null	Key	Default	Extra
codigo	smallint(6)	NO	PRI	NULL	
nombre	varchar(50)	NO		NULL	
fecha_nac	date	YES		NULL	
ciudad	varchar(50)	YES		Las Palmas de GC	

- Insertar un registro indicando la relación entre columnas y valores.
-

4. INSERCIÓN DE DATOS

- EJEMPLO

```
mysql> desc empleados;
```

Field	Type	Null	Key	Default	Extra
codigo	smallint(6)	NO	PRI	NULL	
nombre	varchar(50)	NO		NULL	
fecha_nac	date	YES		NULL	
ciudad	varchar(50)	YES		Las Palmas de GC	

- Insertar un registro indicando la relación entre columnas y valores.

```
INSERT INTO empleados (codigo, fecha_nac, ciudad) VALUES (1, '1992-09-04', 'Néstor', 'Las Palmas de GC');
```

4. INSERCIÓN DE DATOS

- EJEMPLO

```
mysql> desc empleados;
```

Field	Type	Null	Key	Default	Extra
codigo	smallint(6)	NO	PRI	NULL	
nombre	varchar(50)	NO		NULL	
fecha_nac	date	YES		NULL	
ciudad	varchar(50)	YES		Las Palmas de GC	

- Insertar un registro indicando la relación entre columnas y valores.
- No introducir el campo ciudad – ¿Qué valor tomará?

4. INSERCIÓN DE DATOS

- EJEMPLO

```
mysql> desc empleados;
```

Field	Type	Null	Key	Default	Extra
codigo	smallint(6)	NO	PRI	NULL	
nombre	varchar(50)	NO		NULL	
fecha_nac	date	YES		NULL	
ciudad	varchar(50)	YES		Las Palmas de GC	

- Insertar un registro indicando la relación entre columnas y valores.
- No introducir el campo ciudad – ¿Qué valor tomará? **Las Palmas de GC**

```
INSERT INTO empleados (codigo, fecha_nac, nombre) VALUES (1, '1995-11-29', 'Paco');
```

4. INSERCIÓN DE DATOS

- Inserciones
 - Las columnas a las que damos valores se identifican por su nombre
`INSERT INTO empleados (codigo, fecha_nac, nombre) VALUES (1, '1995-11-29', 'Paco');`
 - La asociación columna-valor es **por posición**
 - código - 1
 - fecha_nac - '1995-11-29'
 - nombre - 'Paco'
 - Los valores que se dan a las columnas deben coincidir con el tipo de dato de la columna.
 - Los valores constantes de tipo carácter o fecha deben ir entre comillas simples.
 - Las columnas para las que no damos valores aparecen como NULL
-

4. INSERCIÓN DE DATOS

- EJEMPLO

- ¿Qué va a pasar?

```
INSERT INTO empleados (codigo, nombre, ciudad) VALUES (3, 'Tarugo', 'Telde');
```

Field	Type	Null	Key	Default	Extra
codigo	smallint(6)	NO	PRI	NULL	
nombre	varchar(50)	NO		NULL	
fecha_nac	date	YES		NULL	
ciudad	varchar(50)	YES		Las Palmas de Gran Canaria	

4. INSERCIÓN DE DATOS

- EJEMPLO

Field	Type	Null	Key	Default	Extra
codigo	smallint(6)	NO	PRI	NULL	
nombre	varchar(50)	NO		NULL	
fecha_nac	date	YES		NULL	
ciudad	varchar(50)	YES		Las Palmas de Gran Canaria	

- ¿Qué va a pasar?

```
INSERT INTO empleados (codigo, nombre, ciudad) VALUES (3, 'Tarugo', 'Telde');
```

- Las columnas que no se encuentren en la lista recibirán el valor de NULL.

codigo	nombre	fecha_nac	ciudad
1	Néstor	1992-09-04	Las Palmas de GC
2	Paco	1995-11-29	Las Palmas de Gran Canaria
3	Tarugo	NULL	Telde
NULL	NULL	NULL	NULL

4. INSERCIÓN DE DATOS

- EJEMPLO

Field	Type	Null	Key	Default	Extra
codigo	smallint(6)	NO	PRI	NULL	
nombre	varchar(50)	NO		NULL	
fecha_nac	date	YES		NULL	
ciudad	varchar(50)	YES		Las Palmas de Gran Canaria	

- ¿Qué va a pasar?

```
INSERT INTO empleados (codigo, fecha_nac, ciudad)
VALUES (4, 'Tolete', '99-01-01', 'SC de Tenerife');
```

4. INSERCIÓN DE DATOS

- EJEMPLO

Field	Type	Null	Key	Default	Extra
codigo	smallint(6)	NO	PRI	NULL	
nombre	varchar(50)	NO		NULL	
fecha_nac	date	YES		NULL	
ciudad	varchar(50)	YES		Las Palmas de Gran Canaria	

- ¿Qué va a pasar?

```
INSERT INTO empleados (codigo, fecha_nac, ciudad)
VALUES (4, 'Tolete', '99-01-01', 'SC de Tenerife');
```

❌ 395 00:24:00 INSERT INTO empleados (codigo, fecha_nac, ciudad) VALUES (4, 'Tolete',... Error Code: 1136. Column count doesn't match value count at row 1

- Si se construye una sentencia INSERT con más campos en la lista de valores que el número de columnas especificadas (o número de columnas de la tabla) el SGBD informará del error.
-

4. INSERCIÓN DE DATOS

- EJEMPLO

Field	Type	Null	Key	Default	Extra
codigo	smallint(6)	NO	PRI	NULL	
nombre	varchar(50)	NO		NULL	
fecha_nac	date	YES		NULL	
ciudad	varchar(50)	YES		Las Palmas de Gran Canaria	

- ¿Qué va a pasar?

```
INSERT INTO empleados (codigo, fecha_nac, ciudad) VALUES (4, '99-01-01', 'SC de Tenerife');
```

4. INSERCIÓN DE DATOS

- EJEMPLO

Field	Type	Null	Key	Default	Extra
codigo	smallint(6)	NO	PRI	NULL	
nombre	varchar(50)	NO		NULL	
fecha_nac	date	YES		NULL	
ciudad	varchar(50)	YES		Las Palmas de Gran Canaria	

- ¿Qué va a pasar?

```
INSERT INTO empleados (codigo, fecha_nac, ciudad) VALUES (4, '99-01-01', 'SC de Tenerife');
```

- Si en la definición de la columna, la especificamos como NOT NULL, no podemos insertar una fila con valor nulo en esa columna.

```
mysql> INSERT INTO emple (codigo, fecha_nac, ciudad) VALUES (3, '99-01-01', 'SC de Tenerife');  
ERROR 1364 (HY000): Field 'nombre' doesn't have a default value  
mysql>
```

ERROR: el valor nulo en la columna «nombre» de la relación «empleados» viola la restricción “not-null”
La fila que falla contiene (4, null, 1999-01-04, SC de Tenerife).

SQL state: 23502

Detail: La fila que falla contiene (4, null, 1999-01-04, SC de Tenerife).

4. INSERCIÓN DE DATOS

- EJEMPLO

Field	Type	Null	Key	Default	Extra
codigo	smallint(6)	NO	PRI	NULL	
nombre	varchar(50)	NO		NULL	
fecha_nac	date	YES		NULL	
ciudad	varchar(50)	YES		Las Palmas de Gran Canaria	

- ¿Qué va a pasar?

```
INSERT INTO empleados VALUES (4, 'Tolete', '99-01-01', 'SC de Tenerife');
```

- Si no indicamos las columnas, el orden de introducción de los datos será el que tenga la tabla definida.

	codigo	nombre	fecha_nac	ciudad
1		Néstor	1992-09-04	Las Palmas de GC
2		Paco	1995-11-29	Las Palmas de Gran Canaria
3		Tarugo	NULL	Telde
4		Tolete	1999-01-01	SC de Tenerife
	NULL	NULL	NULL	NULL

4. INSERCIÓN DE DATOS

- EJEMPLO

Field	Type	Null	Key	Default	Extra
codigo	smallint(6)	NO	PRI	NULL	
nombre	varchar(50)	NO		NULL	
fecha_nac	date	YES		NULL	
ciudad	varchar(50)	YES		Las Palmas de Gran Canaria	

- Para la cuarta columna (especificando) elegimos que tome el valor por defecto.

4. INSERCIÓN DE DATOS

- EJEMPLO

Field	Type	Null	Key	Default	Extra
codigo	smallint(6)	NO	PRI	NULL	
nombre	varchar(50)	NO		NULL	
fecha_nac	date	YES		NULL	
ciudad	varchar(50)	YES		Las Palmas de Gran Canaria	

- Para la cuarta columna (especificando) elegimos que tome el valor por defecto.

```
INSERT INTO empleados VALUES (5, 'Simplón', '89-01-01', DEFAULT);
```

codigo	nombre	fecha_nac	ciudad
1	Néstor	1992-09-04	Las Palmas de GC
2	Paco	1995-11-29	Las Palmas de Gran Canaria
3	Tarugo	NULL	Telde
4	Tolete	1999-01-01	SC de Tenerife
5	Simplón	1989-01-01	Las Palmas de Gran Canaria
NULL	NULL	NULL	NULL

EJERCICIO 1

- Insertar la siguiente información en la tabla clientes:
 - Manuel Pérez con DNI 87654321X y población por defecto

```
mysql> describe clientes;
```

Field	Type	Null	Key	Default	Extra
dni_cliente	varchar(9)	NO	PRI	NULL	
nombre	varchar(40)	NO		NULL	
direccion	varchar(40)	YES		NULL	
poblacion	varchar(40)	YES		Madrid	
cp	varchar(5)	YES		NULL	
fecha_nac	date	YES		NULL	

- INSERT INTO nombretabla [(nombre_columna,)] VALUES ({valor | DEFAULT});
-

EJERCICIO 1

- Insertar la siguiente información en la tabla clientes:
 - Manuel Pérez con DNI 87654321X y población por defecto

```
mysql> describe clientes;
```

Field	Type	Null	Key	Default	Extra
dni_cliente	varchar(9)	NO	PRI	NULL	
nombre	varchar(40)	NO		NULL	
direccion	varchar(40)	YES		NULL	
poblacion	varchar(40)	YES		Madrid	
cp	varchar(5)	YES		NULL	
fecha_nac	date	YES		NULL	

```
INSERT INTO clientes (dni_cliente, nombre, poblacion)
VALUES ('87654321X', 'Manuel Pérez', DEFAULT);
```

#	dni_cliente	nombre	direccion	poblacion	cp	fecha_nac
1	87654321X	Manuel Pérez	NULL	Madrid	NULL	NULL

EJERCICIO 2

- Insertar la siguiente información en la tabla películas sin indicar el nombre de las columnas:
 - Cod:1515, El secreto de Thomas Crown, 3 September 1999, 7 en stock, 10 €.

```
mysql> describe peliculas;
```

Field	Type	Null	Key	Default	Extra
cod_pelicula	int(11)	NO	PRI	NULL	
nombre	varchar(40)	NO		NULL	
fecha_alta	date	YES		2006-01-01	
stock	int(11)	YES		10	
precio	int(11)	NO		NULL	

- INSERT INTO nombretabla [(nombre_columna,)] VALUES ({valor | DEFAULT});
-

EJERCICIO 2

- Insertar la siguiente información en la tabla películas sin indicar el nombre de las columnas:
 - Cod:1515, El secreto de Thomas Crown, 3 September 1999, 7 en stock, 10 €.

```
mysql> describe peliculas;
```

Field	Type	Null	Key	Default	Extra
cod_pelicula	int(11)	NO	PRI	NULL	
nombre	varchar(40)	NO		NULL	
fecha_alta	date	YES		2006-01-01	
stock	int(11)	YES		10	
precio	int(11)	NO		NULL	

```
INSERT INTO peliculas  
VALUES ('1515', 'El secreto de Thomas Crown', '1999-09-03', 7, 10 );
```

#	cod_pelicula	nombre	fecha_alta	stock	precio
1	1515	El secreto de Thomas Crown	1999-09-03	7	10

EJERCICIO 3

- Insertar la siguiente información en la tabla películas:
 - Cod:1520, Mars Attacks!

```
mysql> describe peliculas;
```

Field	Type	Null	Key	Default	Extra
cod_pelicula	int(11)	NO	PRI	NULL	
nombre	varchar(40)	NO		NULL	
fecha_alta	date	YES		2006-01-01	
stock	int(11)	YES		10	
precio	int(11)	NO		NULL	

- INSERT INTO nombretabla [(nombre_columna,)] VALUES ({valor | DEFAULT});
-

EJERCICIO 3

- Insertar la siguiente información en la tabla películas:
 - Cod:1520, Mars Attacks!

```
mysql> describe peliculas;
```

Field	Type	Null	Key	Default	Extra
cod_pelicula	int(11)	NO	PRI	NULL	
nombre	varchar(40)	NO		NULL	
fecha_alta	date	YES		2006-01-01	
stock	int(11)	YES		10	
precio	int(11)	NO		NULL	

```
INSERT INTO peliculas (cod_pelicula, nombre)  
VALUES ('1520', 'Mars Attacks!');
```

EJERCICIO 3

¡NO!

- Insertar la siguiente información en la tabla películas:
 - Cod:1520, Mars Attacks!

```
mysql> describe peliculas;
```

Field	Type	Null	Key	Default	Extra
cod_pelicula	int(11)	NO	PRI	NULL	
nombre	varchar(40)	NO		NULL	
fecha_alta	date	YES		2006-01-01	
stock	int(11)	YES		10	
precio	int(11)	NO		NULL	

```
INSERT INTO peliculas (cod_pelicula, nombre)
VALUES ('1520', 'Mars Attacks!');
```



EJERCICIO 3

- Insertar la siguiente información en la tabla películas:
 - Cod:1520, Mars Attacks! – **Tendríamos que consultar el precio**

```
mysql> describe peliculas;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default    | Extra |
+-----+-----+-----+-----+-----+-----+
| cod_pelicula | int(11)       | NO   | PRI | NULL       |       |
| nombre      | varchar(40)   | NO   |     | NULL       |       |
| fecha_alta  | date          | YES  |     | 2006-01-01 |       |
| stock       | int(11)       | YES  |     | 10         |       |
| precio      | int(11)       | NO   |     | NULL       |       |
+-----+-----+-----+-----+-----+-----+
```

```
INSERT INTO peliculas (cod_pelicula, nombre, precio)
VALUES ('1520', 'Mars Attacks!', 10 );
```

#	cod_pelicula	nombre	fecha_alta	stock	precio
6	1520	Mars Attacks!	2006-01-01	10	10



EJERCICIO 4

- Insertar la siguiente información en la tabla películas:
 - Cod:1520, Mars Attacks!, precio 10€
 - Para poner a NULL una columna con valores por defecto:

```
mysql> describe peliculas;
```

Field	Type	Null	Key	Default	Extra
cod_pelicula	int(11)	NO	PRI	NULL	
nombre	varchar(40)	NO		NULL	
fecha_alta	date	YES		2006-01-01	
stock	int(11)	YES		10	
precio	int(11)	NO		NULL	

```
INSERT INTO peliculas (cod_pelicula, nombre, precio)  
VALUES ('1520', 'Mars Attacks!', 10 );
```

#	cod_pelicula	nombre	fecha_alta	stock	precio
6	1520	Mars Attacks!	2006-01-01	10	10

EJERCICIO 4

```
mysql> describe peliculas;
```

Field	Type	Null	Key	Default	Extra
cod_pelicula	int(11)	NO	PRI	NULL	
nombre	varchar(40)	NO		NULL	
fecha_alta	date	YES		2006-01-01	
stock	int(11)	YES		10	
precio	int(11)	NO		NULL	

- Insertar la siguiente información en la tabla películas:
 - Cod:1520, Mars Attacks!, precio 10€
 - Para poner a NULL una columna con valores por defecto:

```
INSERT INTO peliculas (cod_pelicula, nombre, precio)
VALUES ('1520', 'Mars Attacks!', 10 );
```

#	cod_pelicula	nombre	fecha_alta	stock	precio
6	1520	Mars Attacks!	2006-01-01	10	10

```
INSERT INTO peliculas
VALUES ('1521', 'El secreto de Thomas Crown', NULL, NULL, 10 );
```

#	cod_pelicula	nombre	fecha_alta	stock	precio
7	1521	El secreto de Thomas Crown	NULL	NULL	10

4. INSERCIÓN DE DATOS

- La sentencia INSERT EXTENDIDA

INSERT INTO empleados **VALUES**

```
(6, 'Papanata', '89-08-01', 'Telde'),  
(7, 'Sopabobo', '84-08-01', DEFAULT),  
(8, 'Chafalmeja', '89-08-01', 'Vecindario');
```

Field	Type	Null	Key	Default	Extra
codigo	smallint(6)	NO	PRI	NULL	
nombre	varchar(50)	NO		NULL	
fecha_nac	date	YES		NULL	
ciudad	varchar(50)	YES		Las Palmas de Gran Canaria	

	codigo	nombre	fecha_nac	ciudad
1		Néstor	1992-09-04	Las Palmas de GC
2		Paco	1995-11-29	Las Palmas de Gran Canaria
3		Tarugo	NULL	Telde
4		Tolete	1999-01-01	SC de Tenerife
5		Simplón	1989-01-01	Las Palmas de Gran Canaria
6		Papanata	1989-08-01	Telde
7		Sopabobo	1984-08-01	Las Palmas de Gran Canaria
8		Chafalmeja	1989-08-01	Vecindario
	NULL	NULL	NULL	NULL

4. INSERCIÓN DE DATOS

- Una variante de la sentencia INSERT consiste en utilizar la sentencia SELECT para obtener un conjunto de datos y, posteriormente, insertarlos en la tabla.
 - `INSERT INTO nombretabla [(nombre_columna, ....)] SELECT [(nombre_columna, ....)] FROM nombre_tabla;`
 - La sentencia SELECT puede ser tan compleja como se desee, usando filtros, agrupaciones, ordenaciones, etc.
 - Cuando no indican columnas, la sentencia SELECT debe devolver tantas columnas como columnas tenga la tabla donde se introduce la información.
-

4. INSERCIÓN DE DATOS

- EJEMPLO

- Inserta en una tabla Backup todos los empleados.

```
INSERT INTO backupEmpleados  
SELECT * FROM empleados;
```

DML

- En este ejemplo la tabla backupEmpleados tiene una estructura idéntica a la tabla empleados.

- CREATE TABLE backupEmpleados LIKE empleados;

- Alternativas al insert de todo lo de una tabla en otra:

- CREATE TABLE backupEmpleados SELECT * FROM empleados;

DDL

4. INSERCIÓN DE DATOS

- EJEMPLO

```
32 • SELECT * FROM empleados;
```

Result Grid				
Filter Rows:				
Edit: 				
	codigo	nombre	fecha_nac	ciudad
1	1	Néstor	1992-09-04	Las Palmas de GC
2	2	Paco	1995-11-29	Las Palmas de Gran Canaria
3	3	Tarugo	NULL	Telde
4	4	Tolete	1999-01-01	SC de Tenerife
5	5	Simplón	1989-01-01	Las Palmas de Gran Canaria
6	6	Papanata	1989-08-01	Telde
7	7	Sopabobo	1984-08-01	Las Palmas de Gran Canaria
8	8	Chafalmeja	1989-08-01	Vecindario
	NULL	NULL	NULL	NULL

```
34 • INSERT INTO BackUpEmpleados SELECT * FROM empleados;
```

```
35 • SELECT * FROM BackUpEmpleados;
```

Result Grid				
Filter Rows:				
Edit: 				
	codigo	nombre	fecha_nac	ciudad
1	1	Néstor	1992-09-04	Las Palmas de GC
2	2	Paco	1995-11-29	Las Palmas de Gran Canaria
3	3	Tarugo	NULL	Telde
4	4	Tolete	1999-01-01	SC de Tenerife
5	5	Simplón	1989-01-01	Las Palmas de Gran Canaria
6	6	Papanata	1989-08-01	Telde
7	7	Sopabobo	1984-08-01	Las Palmas de Gran Canaria
8	8	Chafalmeja	1989-08-01	Vecindario
	NULL	NULL	NULL	NULL

EJERCICIO 5

Field	Type	Null	Key	Default	Extra
codigo	smallint(6)	NO	PRI	NULL	
nombre	varchar(50)	NO		NULL	
fecha_nac	date	YES		NULL	
ciudad	varchar(50)	YES		Las Palmas de Gran Canaria	

- Crea una tabla llamada empleadosTelde con todos los empleados de la tabla empleados que sean de Telde.

- Tabla Empleados:

codigo	nombre	fecha_nac	ciudad
1	Néstor	1992-09-04	Las Palmas de GC
2	Paco	1995-11-29	Las Palmas de Gran Canaria
3	Tarugo	NULL	Telde
4	Tolete	1999-01-01	SC de Tenerife
5	Simplón	1989-01-01	Las Palmas de Gran Canaria
6	Papanata	1989-08-01	Telde
7	Sopabobo	1984-08-01	Las Palmas de Gran Canaria
8	Chafalmeja	1989-08-01	Vecindario
NULL	NULL	NULL	NULL

EJERCICIO 5

- Crea una tabla llamada empleadosTelde con todos los empleados de la tabla empleados que sean de Telde.

```
INSERT INTO empleadosTelde SELECT codigo, nombre, fecha_nac FROM empleados WHERE ciudad='Telde';
```

```
CREATE TABLE empleadosTelde (  
  codigo INT NOT NULL PRIMARY KEY,  
  nombre VARCHAR(50) not null,  
  fecha_nac DATE  
);
```

```
13 • SELECT * FROM empleadosTelde;
```

Result Grid		
Filter Rows:		
codigo	nombre	fecha_nac
3	Tarugo	NULL
6	Papanata	1989-08-01
NULL	NULL	NULL

EJERCICIO 6

- Inserta en la tabla test todos los clientes que:
 - Sean de Las Palmas de Gran Canaria
 - Su nombre tenga como segundo carácter una a y termine en O
 - Hayan nacido antes de 1955
- Se deben cumplir todas las condiciones a la vez.

```
mysql> desc clientes;
```

Field	Type	Null	Key	Default	Extra
dni_cliente	varchar(9)	NO	PRI	NULL	
nombre	varchar(40)	NO		NULL	
direccion	varchar(40)	YES		NULL	
poblacion	varchar(40)	YES		Madrid	
cp	varchar(5)	YES		NULL	
fecha_nac	date	YES		NULL	

```
INSERT INTO nombretabla [(nombre_columna, ....)]  
SELECT [(nombre_columna, ....)] FROM nombre_tabla;
```

```
mysql> desc test;
```

Field	Type	Null	Key	Default	Extra
dni_cliente	varchar(9)	NO	PRI	NULL	
nombre	varchar(40)	NO		NULL	
direccion	varchar(40)	YES		NULL	

EJERCICIO 6

- Inserta en la tabla test todos los clientes que:

- Sean de Las Palmas de Gran Canaria
- Su nombre tenga como segundo carácter una a y termine en O
- Hayan nacido antes de 1955

- Se deben cumplir todas las condiciones a la vez.

```
INSERT INTO test SELECT dni_cliente, nombre, direccion from clientes
where nombre like '_a%'
and poblacion='Las Palmas de Gran Canaria'
and year(fecha_nac)<1955;
```

```
mysql> select * from test;
+-----+-----+-----+
| dni_cliente | nombre | direccion |
+-----+-----+-----+
| 56987412X   | Paco   | C/La Calva, 25 |
+-----+-----+-----+
1 row in set (0,00 sec)
```

```
mysql> desc clientes;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| dni_cliente | varchar(9) | NO   | PRI | NULL    |       |
| nombre      | varchar(40) | NO   |     | NULL    |       |
| direccion   | varchar(40) | YES  |     | NULL    |       |
| poblacion   | varchar(40) | YES  |     | Madrid  |       |
| cp          | varchar(5)  | YES  |     | NULL    |       |
| fecha_nac   | date       | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
```

```
INSERT INTO nombretabla [(nombre_columna, ....)]
SELECT [(nombre_columna, ....)] FROM nombre_tabla;
```

```
mysql> desc test;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| dni_cliente | varchar(9) | NO   | PRI | NULL    |       |
| nombre      | varchar(40) | NO   |     | NULL    |       |
| direccion   | varchar(40) | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
```

EJERCICIO 7

- Inserta en la tabla backup_pelis el código, nombre y precio (de la tabla películas) de todas películas con un stock inferior a 15.

```
mysql> desc backup_pelis;
```

Field	Type	Null	Key	Default	Extra
cod_pelicula	int(11)	NO	PRI	NULL	
nombre	varchar(40)	NO		NULL	
fecha_alta	date	YES		2006-01-01	
stock	int(11)	YES		10	
precio	int(11)	NO		NULL	

```
INSERT INTO nombretabla [(nombre_columna, ....)]  
SELECT [(nombre_columna, ....)] FROM nombre_tabla;
```

EJERCICIO 7

- Inserta en la tabla backup_pelis el código, nombre y precio (de la tabla películas) de todas películas con un stock inferior a 15.

```
mysql> desc backup_pelis;
```

Field	Type	Null	Key	Default	Extra
cod_pelicula	int(11)	NO	PRI	NULL	
nombre	varchar(40)	NO		NULL	
fecha_alta	date	YES		2006-01-01	
stock	int(11)	YES		10	
precio	int(11)	NO		NULL	

```
INSERT INTO backup_pelis (cod_pelicula, nombre, precio)
SELECT cod_pelicula, nombre, precio
FROM peliculas
WHERE stock < 15;
```

```
INSERT INTO nombretabla [(nombre_columna, ....)]
SELECT [(nombre_columna, ....)] FROM nombre_tabla;
```

EJERCICIO 8

- Insertar en la tabla jobs un nuevo registro con:
 - job_id = REP y job_title = Repartidor
 - El salario máximo y mínimo será el mismo que el del puesto de trabajo Sales Manager.

```
SELECT column_name, data_type, is_nullable, column_default
FROM information_schema.columns
WHERE table_schema = 'public'
  AND table_name = 'jobs'
ORDER BY ordinal_position;
```

#	Field	Type	Null	Key	Default	Extra
1	job_id	varchar(10)	NO	PRI	HULL	
2	job_title	varchar(35)	NO		HULL	
3	min_salary	float	YES		HULL	
4	max_salary	float	YES		HULL	

- SELECT * FROM jobs WHERE job_title = 'Sales Manager';

#	job_id	job_title	min_salary	max_salary
1	SAL	Sales Manager	1200	1700

EJERCICIO 8

- Insertar en la tabla jobs un nuevo registro con:
 - job_id = REP y job_title = Repartidor
 - El salario máximo y mínimo será el mismo que el del puesto de trabajo Sales Manager.
- `SELECT * FROM jobs WHERE job_title = 'Sales Manager';`

```
INSERT INTO jobs
SELECT 'REP', 'Repartidor', min_salary, max_salary
FROM jobs WHERE job_title='Sales Manager';
```

#	job_id	job_title	min_salary	max_salary
1	SAL	Sales Manager	1200	1700

#	job_id	job_title	min_salary	max_salary
1	REP	Repartidor	1200	1700
2	SAL	Sales Manager	1200	1700

EJERCICIO 9

- Inserta en la tabla películasRetro todas las películas (código, nombre, precio, stock, código de director y nombre del director) cuando las películas se hayan dado de alta antes de 1995.

```
mysql> describe peliculas;
```

Field	Type	Null	Key	Default	Extra
codPelicula	int	NO	PRI	NULL	auto_increment
nombre	varchar(40)	NO		NULL	
fechaAlta	date	YES		NULL	
codDirector	int	YES	MUL	NULL	
stock	tinyint	YES		NULL	
precio	double	NO		NULL	

```
6 rows in set (0,01 sec)
```

```
mysql> describe directores;
```

Field	Type	Null	Key	Default	Extra
codDirector	int	NO	PRI	NULL	auto_increment
nombre	varchar(40)	NO		NULL	
fechaNac	date	YES		NULL	

```
3 rows in set (0,00 sec)
```

codPelicula	nombre	fechaAlta	codDirector	stock	precio
1	El Irlandés	2020-04-22	1	10	31.95
2	Shutter Island	2011-06-14	1	5	28.65
3	El Lobo de Wall Street	2014-08-01	1	15	20.99
4	Gangs of New York	2004-03-21	1	12	19.95
5	Pulp Fiction	1995-06-02	2	4	35.95
6	Kill Bill Vol. 1	2004-10-02	2	7	31.99
7	Kill Bill Vol. 2	2005-12-23	2	6	29.95
8	Death Proof	2007-02-23	2	2	40.55
9	El Padrino	1990-01-23	3	2	45.55
10	El Padrino II	1995-08-23	3	1	40.55
11	Star Wars: Episodio IV	1990-10-25	4	2	39.95
12	Star Wars: Episodio V	1990-10-25	4	1	42.95
13	Star Wars: Episodio VI	1990-10-25	4	2	39.8
14	Jurassic Park	1994-10-12	4	9	39.95
15	Pesadillas antes de Navidad	1994-12-12	5	6	36.95
16	Beetlejuice	1990-09-12	5	3	29.95
17	Big Fish	2004-12-02	5	2	37.95
18	Alicia en el País de las Maravillas	2011-08-12	5	4	27.95
19	Solo en Casa	1991-11-12	6	10	29.95
20	Señora Doubtfire	1995-08-06	6	7	20.95
21	Solo en Casa 2	1993-12-12	6	12	31.95
22	Harry Potter y la Piedra Filosofal	2002-03-21	6	15	25.55
23	Harry Potter y la Cámara Secreta	2003-02-01	6	11	29.95
24	Percy Jackson y el Ladrón del Rayo	2011-05-28	6	10	29.95
25	Matrix	2000-01-28	7	10	38.9
26	Matrix Reloaded	2004-01-01	7	10	38.9
27	Matrix Revolution	2005-01-01	7	10	38.9

EJERCICIO 9

- Inserta en la tabla películasRetro todas las películas (código, nombre, precio, stock, código de director y nombre del director) cuando las películas se hayan dado de alta antes de 1995.

```
CREATE TABLE películasRetro (  
  codPelicula INT NOT NULL AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(40) NOT NULL,  
  precio DOUBLE NOT NULL check(precio<=999.99),  
  stock TINYINT NULL,  
  codDirector int,  
  nombreDirector varchar(40));
```

```
INSERT INTO películasRetro  
SELECT películas.codPelicula, películas.nombre, películas.precio, películas.stock, directores.codDirector, directores.nombre from películas  
  inner join directores on películas.codDirector=directores.codDirector  
 where year(películas.fechaAlta)<1995;
```

```
mysql> select * from películasRetro;
```

codPelicula	nombre	precio	stock	codDirector	nombreDirector
9	El Padrino	45.55	2	3	Francis Ford Coppola
11	Star Wars: Episodio IV	39.95	2	4	George Lucas
12	Star Wars: Episodio V	42.95	1	4	George Lucas
13	Star Wars: Episodio VI	39.8	2	4	George Lucas
14	Jurassic Park	39.95	9	4	George Lucas
15	Pesadillas antes de Navidad	38.95	6	5	Tim Burton
16	Beetlejuice	29.95	3	5	Tim Burton
19	Solo en Casa	29.95	10	6	Chris Columbus
21	Solo en Casa 2	31.95	12	6	Chris Columbus

5. MODIFICACIÓN DE DATOS

- La sentencia UPDATE de SQL permite modificar el contenido de cualquier columna y de cualquier fila de una tabla.
- Su sintaxis es la siguiente:
 - UPDATE nombretabla
 - SET col1=valor1, col2=valor2,...
 - WHERE condición

5. MODIFICACIÓN DE DATOS

- Su sintaxis es la siguiente:
 - UPDATE nombretabla
 - SET col1=valor1, col2=valor2,...
 - WHERE condición
 - La actualización se realiza dando a las columnas col1, col2...los valores valor1, valor2,...
 - Se actualizan todas las filas seleccionadas por el filtro indicado mediante la cláusula WHERE.
 - **Esta cláusula WHERE, es idéntica a la que se ha utilizado para el filtrado de registros en la sentencia SELECT.**
-

5. MODIFICACIÓN DE DATOS

- EJEMPLO

- ANTES

codigo	nombre	fecha_nac	ciudad
1	Néstor	1992-09-04	Las Palmas de GC
2	Paco	1995-11-29	Las Palmas de Gran Canaria
3	Tarugo	NULL	Telde
4	Tolete	1999-01-01	SC de Tenerife
5	Simplón	1989-01-01	Las Palmas de Gran Canaria
6	Papanata	1989-08-01	Telde
7	Sopabobo	1984-08-01	Las Palmas de Gran Canaria
8	Chafalmeja	1989-08-01	Vecindario
NULL	NULL	NULL	NULL

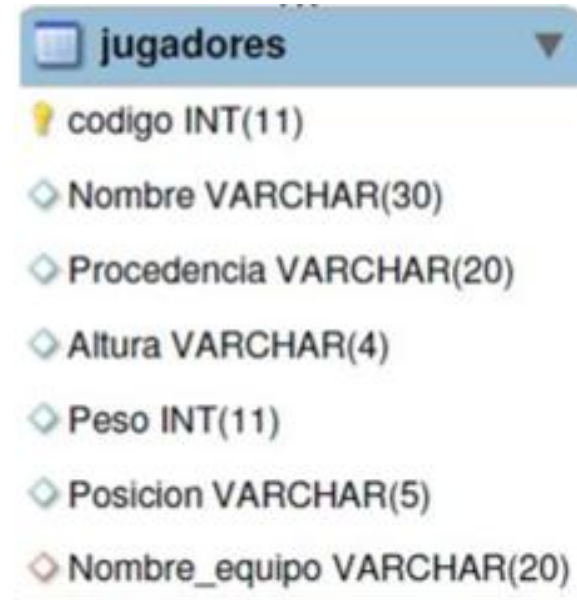
```
UPDATE empleados SET nombre = 'Totorota' where codigo = 5;
```

- DESPUÉS

codigo	nombre	fecha_nac	ciudad
1	Néstor	1992-09-04	Las Palmas de GC
2	Paco	1995-11-29	Las Palmas de Gran Canaria
3	Tarugo	NULL	Telde
4	Tolete	1999-01-01	SC de Tenerife
5	Totorota	1989-01-01	Las Palmas de Gran Canaria
6	Papanata	1989-08-01	Telde
7	Sopabobo	1984-08-01	Las Palmas de Gran Canaria
8	Chafalmeja	1989-08-01	Vecindario
NULL	NULL	NULL	NULL

EJERCICIO 10

- Se desea actualizar el peso de 'Pau Gasol' a 215 libras.
 - UPDATE nombretabla
 - SET col1=valor1, col2=valor2,...
 - WHERE condición

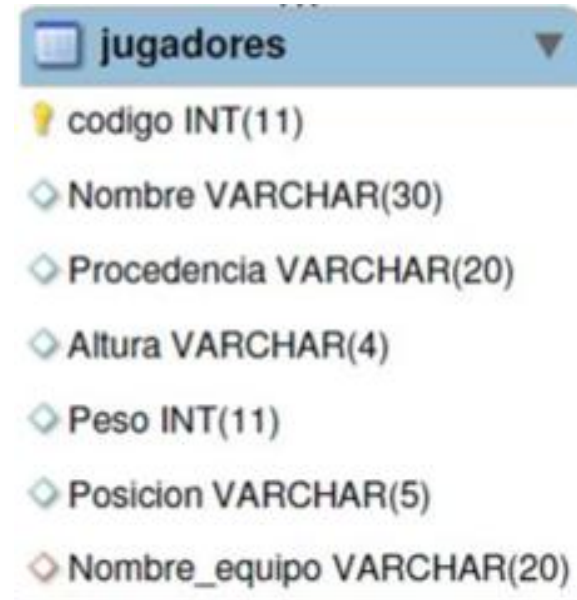


A screenshot of a database table structure for a table named 'jugadores'. The table has the following columns:

jugadores
codigo INT(11)
Nombre VARCHAR(30)
Procedencia VARCHAR(20)
Altura VARCHAR(4)
Peso INT(11)
Posicion VARCHAR(5)
Nombre_equipo VARCHAR(20)

EJERCICIO 10

- Se desea actualizar el peso de 'Pau Gasol' a 215 libras.
 - UPDATE jugadores
 - SET Peso=215
 - WHERE Nombre='Pau Gasol';

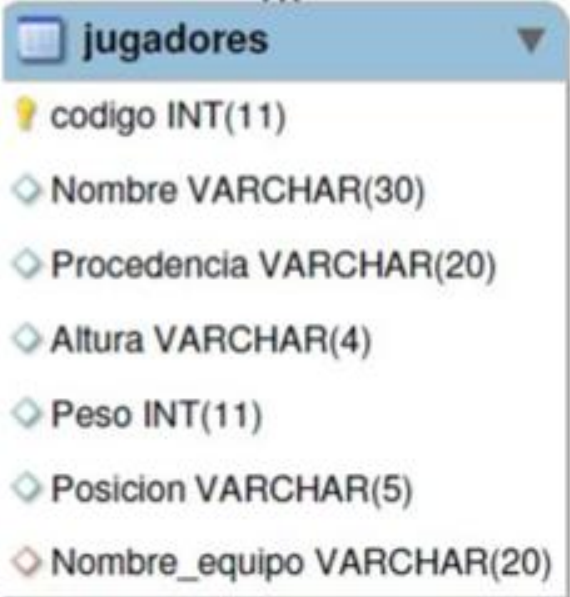


A screenshot of a database table structure for a table named 'jugadores'. The table has the following columns:

jugadores
codigo INT(11)
Nombre VARCHAR(30)
Procedencia VARCHAR(20)
Altura VARCHAR(4)
Peso INT(11)
Posicion VARCHAR(5)
Nombre_equipo VARCHAR(20)

EJERCICIO 11

- Se desea actualizar el equipo de 'Pau Gasol' porque ha fichado por otro equipo, por ejemplo, los 'Knicks'.
 - UPDATE nombretabla
 - SET col1=valor1, col2=valor2,...
 - WHERE condición

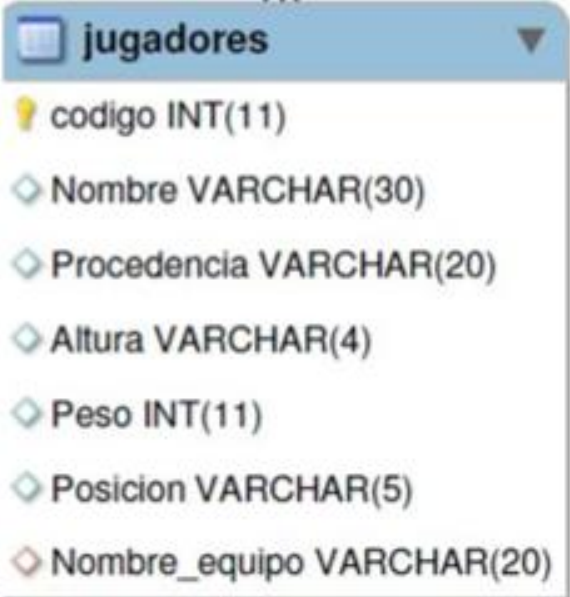


A screenshot of a database table structure for a table named 'jugadores'. The table has the following columns and data types:

codigo	Nombre	Procedencia	Altura	Peso	Posicion	Nombre_equipo
INT(11)	VARCHAR(30)	VARCHAR(20)	VARCHAR(4)	INT(11)	VARCHAR(5)	VARCHAR(20)

EJERCICIO 11

- Se desea actualizar el equipo de 'Pau Gasol' porque ha fichado por otro equipo, por ejemplo, los 'Knicks'.
 - UPDATE jugadores
 - SET Nombre_equipo='Knicks'
 - WHERE Nombre='Pau Gasol';



A screenshot of a database table named 'jugadores'. The table has the following columns:

codigo	Nombre	Procedencia	Altura	Peso	Posicion	Nombre_equipo
INT(11)	VARCHAR(30)	VARCHAR(20)	VARCHAR(4)	INT(11)	VARCHAR(5)	VARCHAR(20)

5. MODIFICACIÓN DE DATOS

- EJEMPLO SIN WHERE

codigo	nombre	fecha_nac	ciudad
1	Néstor	1992-09-04	Las Palmas de GC
2	Paco	1995-11-29	Las Palmas de Gran Canaria
3	Machango	NULL	Telde
4	Machango	1999-01-01	SC de Tenerife
5	Machango	1989-01-01	Las Palmas de Gran Canaria
6	Papanata	1989-08-01	Telde
7	Sopabobo	1984-08-01	Las Palmas de Gran Canaria
8	Chafalmeja	1989-08-01	Vecindario
NULL	NULL	NULL	NULL

- ¿Qué vas a pasar?

```
UPDATE empleados SET nombre = 'Totorota';
```

5. MODIFICACIÓN DE DATOS

- EJEMPLO SIN WHERE

```
UPDATE empleados SET nombre = 'Totorota';
```

ANTES

codigo	nombre	fecha_nac	ciudad
1	Néstor	1992-09-04	Las Palmas de GC
2	Paco	1995-11-29	Las Palmas de Gran Canaria
3	Machango	NULL	Telde
4	Machango	1999-01-01	SC de Tenerife
5	Machango	1989-01-01	Las Palmas de Gran Canaria
6	Papanata	1989-08-01	Telde
7	Sopabobo	1984-08-01	Las Palmas de Gran Canaria
8	Chafalmeja	1989-08-01	Vecindario
NULL	NULL	NULL	NULL

DESPUÉS

codigo	nombre	fecha_nac	ciudad
1	Totorota	1992-09-04	Las Palmas de GC
2	Totorota	1995-11-29	Las Palmas de Gran Canaria
3	Totorota	NULL	Telde
4	Totorota	1999-01-01	SC de Tenerife
5	Totorota	1989-01-01	Las Palmas de Gran Canaria
6	Totorota	1989-08-01	Telde
7	Totorota	1984-08-01	Las Palmas de Gran Canaria
8	Totorota	1989-08-01	Vecindario
NULL	NULL	NULL	NULL

5. MODIFICACIÓN DE DATOS

- Si se omite el filtro o condición (WHERE), el resultado es la modificación de todos los registros de la tabla.
- **Por ejemplo:** para cambiar el peso de los jugadores de la NBA de libras a kilos:
 - UPDATE jugadores SET Peso=Peso*0.4535;

5. MODIFICACIÓN DE DATOS

- UPDATE con SELECT
 - Es posible actualizar registros de una tabla filtrando a través de una subconsulta.
 - **SELECT puede estar contenida en la cláusula WHERE o puede formar parte de SET.**
 - **Cuando la subconsulta (SELECT) forma parte de SET debe seleccionar una única fila y una única columna.**
 - PostgreSQL sí permite actualizar o borrar registros utilizando subconsultas sobre la misma tabla.
-

5. MODIFICACIÓN DE DATOS

- UPDATE con SELECT
 - Formato1:
 - UPDATE nombretabla
 - SET columna1 = (SELECT col1...),
 - columna2 =(SELECT col2...), ...
 - WHERE condición
 - Formato2:
 - UPDATE nombretabla
 - SET columna1=valor1,
 - columna2=valor2 ...
 - WHERE condición=(SELECT...)
-

5. MODIFICACIÓN DE DATOS

- EJEMPLO – FORMATO 1

- La tabla clientes tiene los registros expuestos a continuación:

```
mysql> select * from clientes;
```

dni_cliente	nombre	direccion	poblacion	cp	fecha_nac
12345678M	Marta Torres	calle nº15	Madrid	25003	2006-02-24
87654321X	Cristina Cifuentes	Guanarteme nº18	Madrid	25008	NULL
87654322X	Yanis Manuel	pepito nº 65	Barcelona	22006	NULL
87654323X	Manuel Pérez	Pilar nº 89	Las Palmas de GC	35008	NULL

- Modificar la población de Marta Torres con la población del cliente Yanis Manuel.
 - **TENEMOS DOS TABLAS CLIENTES Y CLIENTES2 (COPIAS EXACTAS)**
-

5. MODIFICACIÓN DE DATOS

- EJEMPLO – FORMATO 1

- La tabla clientes tiene los registros expuestos a continuación:

```
mysql> select * from clientes;
```

dni_cliente	nombre	direccion	poblacion	cp	fecha_nac
12345678M	Marta Torres	calle nº15	Madrid	25003	2006-02-24
87654321X	Cristina Cifuentes	Guanarteme nº18	Madrid	25008	NULL
87654322X	Yanis Manuel	pepito nº 65	Barcelona	22006	NULL
87654323X	Manuel Pérez	Pilar nº 89	Las Palmas de GC	35008	NULL

- Modificar la población de Marta Torres con la población del cliente Yanis Manuel.

```
UPDATE clientes2
SET poblacion =
    (select poblacion FROM clientes
     WHERE nombre='Yanis Manuel')
WHERE nombre='Marta Torres';
```

5. MODIFICACIÓN DE DATOS

- EJEMPLO – FORMATO 2

- La tabla clientes tiene los registros expuestos a continuación:

```
mysql> select * from clientes;
```

dni_cliente	nombre	direccion	poblacion	cp	fecha_nac
12345678M	Marta Torres	calle nº15	Madrid	25003	2006-02-24
87654321X	Cristina Cifuentes	Guanarteme nº18	Madrid	25008	NULL
87654322X	Yanis Manuel	pepito nº 65	Barcelona	22006	NULL
87654323X	Manuel Pérez	Pilar nº 89	Las Palmas de GC	35008	NULL

- Cambiar la población a “Tenerife” y dirección a NULL del cliente que haya comprado la película con código 1520.

```
mysql> select * from compras;
```

codigo_pelicula	dni_cliente	fecha_compra
1520	12345678M	2006-08-09
1515	87654321X	2006-05-09

5. MODIFICACIÓN DE DATOS

- EJEMPLO – FORMATO 2

```
mysql> select * from compras;
```

codigo_pelicula	dni_cliente	fecha_compra
1520	12345678M	2006-08-09
1515	87654321X	2006-05-09

- La tabla clientes tiene los registros expuestos a continuación:

```
mysql> select * from clientes;
```

dni_cliente	nombre	direccion	poblacion	cp	fecha_nac
12345678M	Marta Torres	calle nº15	Madrid	25003	2006-02-24
87654321X	Cristina Cifuentes	Guanarteme nº18	Madrid	25008	NULL
87654322X	Yanis Manuel	pepito nº 65	Barcelona	22006	NULL
87654323X	Manuel Pérez	Pilar nº 89	Las Palmas de GC	35008	NULL

- Cambiar la población a “Tenerife” y dirección a NULL del cliente que haya comprado la película con código 1520.

```
UPDATE clientes2
SET poblacion = 'Tenerife', direccion = NULL
WHERE dni_cliente=
(SELECT dni_cliente FROM compras
WHERE codigo_pelicula=1520);
```

```
mysql> select * from clientes2;
```

dni_cliente	nombre	direccion	poblacion	cp	fecha_nac
12345678M	Marta Torres	NULL	Tenerife	25003	2006-02-24

EJERCICIO 12

- La tabla clientes tiene los registros expuestos a continuación:

```
mysql> select * from clientes;
```

dni_cliente	nombre	direccion	poblacion	cp	fecha_nac
12345678M	Marta Torres	calle nº15	Madrid	25003	2006-02-24
87654321X	Cristina Cifuentes	Guanarteme nº18	Madrid	25008	NULL
87654322X	Yanis Manuel	pepito nº 65	Barcelona	22006	NULL
87654323X	Manuel Pérez	Pilar nº 89	Las Palmas de GC	35008	NULL

- Modificar la dirección del cliente dni=12345678 en la tabla clientes2 con la dirección del cliente Yanis Manuel.

Formato1:

```
UPDATE nombretabla
SET columna1=(SELECT col1...),
    columna2 =(SELECT col2...), ...
WHERE condición
```

Formato2:

```
UPDATE nombretabla
SET columna1=valor1,
    columna2=valor2 ...
WHERE condición=(SELECT...)
```

EJERCICIO 12

- La tabla clientes tiene los registros expuestos a continuación:

```
mysql> select * from clientes;
```

dni_cliente	nombre	direccion	poblacion	cp	fecha_nac
12345678M	Marta Torres	calle nº15	Madrid	25003	2006-02-24
87654321X	Cristina Cifuentes	Guanarteme nº18	Madrid	25008	NULL
87654322X	Yanis Manuel	pepito nº 65	Barcelona	22006	NULL
87654323X	Manuel Pérez	Pilar nº 89	Las Palmas de GC	35008	NULL

- Modificar la dirección del cliente dni=12345678 en la tabla clientes2 con la dirección del cliente Yanis Manuel.

```
UPDATE clientes2
SET direccion =
  (select direccion FROM clientes
   WHERE nombre='Yanis Manuel')
WHERE dni_cliente='12345678M';
```

```
mysql> mysql> select * from clientes2;
```

dni_cliente	nombre	direccion	poblacion
12345678M	Marta Torres	pepito nº 65	Madrid
87654321X	Cristina Cifuentes	Guanarteme nº18	Madrid
87654322X	Yanis Manuel	pepito nº 65	Barcelona
87654323X	Manuel Pérez	Pilar nº 89	Las Palmas de

EJERCICIO 13

- La tabla clientes tiene los registros expuestos a continuación:

```
mysql> select * from clientes;
```

dni_cliente	nombre	direccion	poblacion	cp	fecha_nac
12345678M	Marta Torres	calle nº15	Madrid	25003	2006-02-24
87654321X	Cristina Cifuentes	Guanarteme nº18	Madrid	25008	NULL
87654322X	Yanis Manuel	pepito nº 65	Barcelona	22006	NULL
87654323X	Manuel Pérez	Pilar nº 89	Las Palmas de GC	35008	NULL

- Modificar la dirección de “Cristina Cifuentes” en la tabla clientes2 con la dirección del cliente con dni=12345678.

Formato1:

```
UPDATE nombretabla
SET columna1=(SELECT col1...),
      columna2 =(SELECT col2...), ...
WHERE condición
```

Formato2:

```
UPDATE nombretabla
SET columna1=valor1,
      columna2=valor2 ...
WHERE condición=(SELECT...)
```

EJERCICIO 13

- La tabla clientes tiene los registros expuestos a continuación:

```
mysql> select * from clientes;
```

dni_cliente	nombre	direccion	poblacion	cp	fecha_nac
12345678M	Marta Torres	calle nº15	Madrid	25003	2006-02-24
87654321X	Cristina Cifuentes	Guanarteme nº18	Madrid	25008	NULL
87654322X	Yanis Manuel	pepito nº 65	Barcelona	22006	NULL
87654323X	Manuel Pérez	Pilar nº 89	Las Palmas de GC	35008	NULL

- Modificar la dirección de “Cristina Cifuentes” en la tabla clientes2 con la dirección del cliente con dni=12345678.

```
UPDATE clientes2
SET direccion=
  (SELECT direccion FROM clientes
   WHERE dni_cliente= '12345678M')
WHERE nombre = 'Cristina Cifuentes';
```

```
mysql> select * from clientes2;
```

dni_cliente	nombre	direccion	poblacion
12345678M	Marta Torres	calle nº15	Madrid
87654321X	Cristina Cifuentes	calle nº15	Madrid
87654322X	Yanis Manuel	pepito nº 65	Barcelona
87654323X	Manuel Pérez	Pilar nº 89	Las Palmas

EJERCICIO 14

- Poner el precio de todas las películas un 10% más caro:

```
mysql> select * from peliculas;
```

cod_pelicula	nombre	fecha_alta	stock	precio
1515	El secreto de Thomas Crown	1999-09-03	7	10
1516	El secreto de Thomas Crown	1999-09-03	7	10
1517	El secreto de Thomas Crown	1999-09-03	7	10
1518	El secreto de Thomas Crown	1999-09-03	7	10
1519	El secreto de Thomas Crown	1999-09-03	7	10
1520	Mars Attacks!	2006-01-01	10	10
1521	El secreto de Thomas Crown	NULL	NULL	10

Formato1:

```
UPDATE nombretabla
SET columna1=(SELECT col1...),
      columna2 =(SELECT col2...), ...
WHERE condición
```

Formato2:

```
UPDATE nombretabla
SET columna1=valor1,
    columna2=valor2 ...
WHERE condición=(SELECT...)
```

EJERCICIO 14

- Poner el precio de todas las películas un 10% más caro:

```
mysql> select * from peliculas;
```

cod_pelicula	nombre	fecha_alta	stock	precio
1515	El secreto de Thomas Crown	1999-09-03	7	10
1516	El secreto de Thomas Crown	1999-09-03	7	10
1517	El secreto de Thomas Crown	1999-09-03	7	10
1518	El secreto de Thomas Crown	1999-09-03	7	10
1519	El secreto de Thomas Crown	1999-09-03	7	10
1520	Mars Attacks!	2006-01-01	10	10
1521	El secreto de Thomas Crown	NULL	NULL	10

```
UPDATE peliculas  
SET precio = precio+precio*10/100;
```

```
mysql> select * from peliculas;
```

cod_pelicula	nombre	fecha_alta	stock	precio
1515	El secreto de Thomas Crown	1999-09-03	7	11
1516	El secreto de Thomas Crown	1999-09-03	7	11
1517	El secreto de Thomas Crown	1999-09-03	7	11
1518	El secreto de Thomas Crown	1999-09-03	7	11
1519	El secreto de Thomas Crown	1999-09-03	7	11
1520	Mars Attacks!	2006-01-01	10	11
1521	El secreto de Thomas Crown	NULL	NULL	11

6. ELIMINACIÓN DE DATOS

- En SQL se utiliza la sentencia DELETE para eliminar filas de una tabla.
- Su sintaxis es:
 - DELETE FROM NombreTabla [WHERE condicion]
- El comando DELETE borra los registros seleccionados por el filtro WHERE, que es idéntico al de la sentencia SELECT.

6. ELIMINACIÓN DE DATOS

- En SQL se utiliza la sentencia DELETE para eliminar filas de una tabla.
- Su sintaxis es:
 - DELETE FROM NombreTabla [WHERE condicion]
- El comando DELETE borra los registros seleccionados por el filtro WHERE, que es idéntico al de la sentencia SELECT.
- **Si no especificamos la cláusula WHERE, borraremos TODAS las filas de la tabla.**



6. ELIMINACIÓN DE DATOS

- EJEMPLO
 - Borra todas las filas de la tabla empleados en las que el nombre es Totorota.

codigo	nombre	fecha_nac	ciudad
1	Néstor	1992-09-04	Las Palmas de GC
2	Paco	1995-11-29	Las Palmas de Gran Canaria
3	Tarugo	NULL	Telde
4	Tolete	1999-01-01	SC de Tenerife
5	Totorota	1989-01-01	Las Palmas de Gran Canaria
6	Papanata	1989-08-01	Telde
7	Sopabobo	1984-08-01	Las Palmas de Gran Canaria
8	Chafalmeja	1989-08-01	Vecindario
NULL	NULL	NULL	NULL

6. ELIMINACIÓN DE DATOS

- EJEMPLO
 - Borra todas las filas de la tabla empleados en las que el nombre es Totorota.

codigo	nombre	fecha_nac	ciudad
1	Néstor	1992-09-04	Las Palmas de GC
2	Paco	1995-11-29	Las Palmas de Gran Canaria
3	Tarugo	NULL	Telde
4	Tolete	1999-01-01	SC de Tenerife
5	Totorota	1989-01-01	Las Palmas de Gran Canaria
6	Papanata	1989-08-01	Telde
7	Sopabobo	1984-08-01	Las Palmas de Gran Canaria
8	Chafalmeja	1989-08-01	Vecindario
NULL	NULL	NULL	NULL

```
DELETE FROM empleados WHERE nombre = 'Totorota';
```

6. ELIMINACIÓN DE DATOS

- EJEMPLO
 - Borra todas las filas de la tabla empleados en las que la fecha sea mayor o igual que la fecha más antigua de la tabla empleadosTelde.

codigo	nombre	fecha_nac	ciudad
1	Néstor	1992-09-04	Las Palmas de GC
2	Paco	1995-11-29	Las Palmas de Gran Canaria
3	Tarugo	NULL	Telde
4	Tolete	1999-01-01	SC de Tenerife
5	Totorota	1989-01-01	Las Palmas de Gran Canaria
6	Papanata	1989-08-01	Telde
7	Sopabobo	1984-08-01	Las Palmas de Gran Canaria
8	Chafalmeja	1989-08-01	Vecindario
NULL	NULL	NULL	NULL

codigo	nombre	fecha_nac
3	Tarugo	NULL
6	Papanata	1989-08-01
NULL	NULL	NULL

6. ELIMINACIÓN DE DATOS

- EJEMPLO

- Borra todas las filas de la tabla empleados en las que la fecha sea mayor o igual que la fecha más antigua de la tabla empleadosTelde.

codigo	nombre	fecha_nac	ciudad
1	Néstor	1992-09-04	Las Palmas de GC
2	Paco	1995-11-29	Las Palmas de Gran Canaria
3	Tarugo	NULL	Telde
4	Tolete	1999-01-01	SC de Tenerife
5	Totorota	1989-01-01	Las Palmas de Gran Canaria
6	Papanata	1989-08-01	Telde
7	Sopabobo	1984-08-01	Las Palmas de Gran Canaria
8	Chafalmeja	1989-08-01	Vecindario
NULL	NULL	NULL	NULL

codigo	nombre	fecha_nac
3	Tarugo	NULL
6	Papanata	1989-08-01
NULL	NULL	NULL

```
DELETE FROM empleados WHERE fecha_nac >= (SELECT MIN(fecha_nac) FROM empleadosTelde);
```

EJERCICIO 15

- La tabla clientes se ha copiado en la tabla clientes2.
- Eliminar los clientes cuya dirección empiece por “p”.

```
mysql> select * from clientes;
```

dni_cliente	nombre	direccion	poblacion	cp	fecha_nac
12345678M	Marta Torres	calle nº15	Madrid	25003	2006-02-24
87654321X	Cristina Cifuentes	Guanarteme nº18	Madrid	25008	NULL
87654322X	Yanis Manuel	pepito nº 65	Barcelona	22006	NULL
87654323X	Manuel Pérez	Pilar nº 89	Las Palmas de GC	35008	NULL

DELETE FROM NombreTabla [WHERE condicion]

EJERCICIO 15

- La tabla clientes se ha copiado en la tabla clientes2.
- Eliminar los clientes cuya dirección empiece por “p”.

```
DELETE FROM clientes2  
WHERE direccion LIKE 'p%';
```

```
mysql> select * from clientes;  
+-----+-----+-----+-----+-----+-----+  
| dni_cliente | nombre      | direccion      | poblacion | cp    | fecha_nac |  
+-----+-----+-----+-----+-----+-----+  
| 12345678M   | Marta Torres | calle nº15     | Madrid    | 25003 | 2006-02-24 |  
| 87654321X   | Cristina Cifuentes | Guanarteme nº18 | Madrid    | 25008 | NULL        |  
| 87654322X   | Yanis Manuel | pepito nº 65   | Barcelona | 22006 | NULL        |  
| 87654323X   | Manuel Pérez | Pilar nº 89    | Las Palmas de GC | 35008 | NULL        |  
+-----+-----+-----+-----+-----+-----+
```

```
mysql> select * from clientes2;  
+-----+-----+-----+-----+  
| dni_cliente | nombre      | direccion      | poblacion |  
+-----+-----+-----+-----+  
| 12345678M   | Marta Torres | calle nº15     | Madrid    |  
| 87654321X   | Cristina Cifuentes | Guanarteme nº18 | Madrid    |  
+-----+-----+-----+-----+
```

7. BORRADO Y MODIFICACIÓN DE REGISTROS CON RELACIONES

- Registros con relaciones
 - **Hay que tener en cuenta que no siempre se pueden borrar o modificar datos de una tabla.**
 - Considérese, por ejemplo, que un cliente llama a una empresa pidiendo darse de baja como cliente, pero el cliente tiene algunos pagos pendientes.
 - Si el operador de la BBDD intenta eliminar el registro (DELETE), el SGBD debería informar de que no es posible eliminar ese registro puesto que hay registros relacionados.



7. BORRADO Y MODIFICACIÓN DE REGISTROS CON RELACIONES

- Registros con relaciones
 - O por ejemplo, se desea cambiar (UPDATE) el nombre de un equipo de la NBA (que es su clave primaria)
 - ¿Qué sucede con los jugadores?
 - También habrá que cambiar el nombre del equipo de los jugadores, puesto que el campo NombreEquipo es una clave foránea.
-

7. BORRADO Y MODIFICACIÓN DE REGISTROS CON RELACIONES

- En este punto, hay que recordar las cláusulas REFERENCES de la sentencia CREATE TABLE para crear las relaciones de clave foránea-clave primaria de alguna columna de una tabla:
 - [CONSTRAINT [symbol]] FOREIGN KEY (ColumnaClaveForánea) REFERENCES NombreTabla(NombreColumna)
 - [ON DELETE opción] | [ON UPDATE opción]
 - opción: CASCADE | SET NULL | RESTRICT
 - Las opciones ON DELETE y ON UPDATE indican qué ocurre en la tabla que estamos creando si se intenta modificar o eliminar filas en la tabla principal que están relacionadas con estas.
-

7. BORRADO Y MODIFICACIÓN DE REGISTROS CON RELACIONES

- FK – Referencias
 - Las opciones pueden ser:
 - RESTRICT
 - Es el **comportamiento por defecto**
 - Impide realizar modificaciones que atentan contra la integridad referencial.
 - Significa que no se puede realizar la acción si hay filas relacionadas en la tabla secundaria.
 - CASCADE:
 - Borra los registros de la tabla dependiente cuando se borra el registro de la tabla principal (en una sentencia DELETE)
 - Actualiza el valor de la clave secundaria cuando se actualiza el valor de la clave referenciada (en una sentencia UPDATE).
-

7. BORRADO Y MODIFICACIÓN DE REGISTROS CON RELACIONES

- FK – Referencias
 - Las opciones pueden ser:
 - SET NULL:
 - Establece a NULL el valor de la clave secundaria cuando se elimina el registro en la tabla principal o se modifica el valor del campo referenciado.
 - NO ACTION:
 - En MySQL es equivalente a RESTRICT

7. BORRADO Y MODIFICACIÓN DE REGISTROS CON RELACIONES

- EJEMPLO RESTRICT

```
#dos tablas relacionadas en mysql
#han de ser innodb para soportar FOREIGN KEYS
CREATE TABLE clientes (
  dni varchar(15) PRIMARY KEY,
  nombre varchar(50),
  dirección varchar(50)
) engine=innodb;
CREATE TABLE pagos_pendientes(
  dni varchar(15),
  importe double,
  FOREIGN KEY(dni) REFERENCES clientes(dni)
  ON DELETE RESTRICT
  ON UPDATE RESTRICT
) engine=innodb;
#un cliente y dos pagos pendientes
INSERT INTO clientes
VALUES ('5555672L', 'Pepe Cifuentes', 'C/Los almendros,23');
INSERT INTO pagos_pendientes VALUES ('5555672L', 500);
INSERT INTO pagos_pendientes VALUES ('5555672L', 234.5);
```

7. BORRADO Y MODIFICACIÓN DE REGISTROS CON RELACIONES

- EJEMPLO RESTRICT

- DELETE FROM clientes WHERE dni='5555672L';
- UPDATE clientes SET dni='5555555L' WHERE dni='5555672L';
- DROP TABLE clientes;

```
#dos tablas relacionadas en mysql
#han de ser innodb para soportar FOREIGN KEYS
CREATE TABLE clientes (
  dni varchar(15) PRIMARY KEY,
  nombre varchar(50),
  dirección varchar(50)
) engine=innodb;
CREATE TABLE pagos_pendientes(
  dni varchar(15),
  importe double,
  FOREIGN KEY(dni) REFERENCES clientes(dni)
  ON DELETE RESTRICT
  ON UPDATE RESTRICT
) engine=innodb;
#un cliente y dos pagos pendientes
INSERT INTO clientes
VALUES ('5555672L', 'Pepé Cifuentes', 'C/Los almendros,23');
INSERT INTO pagos_pendientes VALUES ('5555672L', 500);
INSERT INTO pagos_pendientes VALUES ('5555672L', 234.5);
```

7. BORRADO Y MODIFICACIÓN DE REGISTROS CON RELACIONES

- EJEMPLO RESTRICT

```
mysql> DELETE FROM clientes WHERE dni='5555672L';  
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (`cliente_pagos`.`pagos_pendientes`, CONSTRAINT `pagos_pendientes_ibfk_1` FOREIGN KEY (`dni`) REFERENCES `clientes` (`dni`))
```

- DELETE FROM clientes WHERE dni='5555672L';

```
mysql> UPDATE clientes SET dni='55555555L'  
-> WHERE dni='5555672L';  
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (`cliente_pagos`.`pagos_pendientes`, CONSTRAINT `pagos_pendientes_ibfk_1` FOREIGN KEY (`dni`) REFERENCES `clientes` (`dni`))
```

- UPDATE clientes SET dni='55555555L' WHERE dni="5555672L";

```
mysql> DROP TABLE clientes;  
ERROR 1217 (23000): Cannot delete or update a parent row: a foreign key constraint fails
```

- DROP TABLE clientes;
-

7. BORRADO Y MODIFICACIÓN DE REGISTROS CON RELACIONES

- EJEMPLO CASCADE
 - Sin embargo, si la creación de la relación estuviese personalizada con las opciones ON UPDATE CASCADE y ON DELETE CASCADE, el comportamiento sería el siguiente:
 - Partimos del ejemplo anterior, pero como si se hubiera creado la tabla pagos_pendientes de la siguiente manera:

```
CREATE TABLE pagos_pendientes(  
    dni varchar(15),  
    importe double,  
    FOREIGN KEY(dni) REFERENCES clientes(dni)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
) engine=innodb;
```

7. BORRADO Y MODIFICACIÓN DE REGISTROS CON RELACIONES

- EJEMPLO CASCADE

- DELETE FROM clientes WHERE dni='5555672L'; (Se borra el cliente...)

```
mysql> DELETE FROM clientes WHERE dni='5555672L';  
Query OK, 1 row affected (0,00 sec)
```

- Además, se verifica que ha borrado en cascada sus pagos pendientes.

```
mysql> SELECT * FROM pagos_pendientes;  
Empty set (0,00 sec)
```



7. BORRADO Y MODIFICACIÓN DE REGISTROS CON RELACIONES

- EJEMPLO CASCADE

- Si en lugar de borrar el cliente, se hubiera cambiado el dni.

```
mysql> UPDATE clientes SET dni='55555555L'
-> WHERE dni='5555672L';
Query OK, 1 row affected (0,02 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

- UPDATE clientes SET dni='55555555L' WHERE dni='5555672L';

- Ha cambiado el dni de los pagos en cascada

```
mysql> SELECT * FROM pagos_pendientes;
+-----+-----+
| dni    | importe |
+-----+-----+
| 55555555L | 500    |
| 55555555L | 234.5  |
+-----+-----+
```

7. BORRADO Y MODIFICACIÓN DE REGISTROS CON RELACIONES

- EJEMPLO SET NULL
 - Si la creación de la relación estuviese personalizada con las opciones ON UPDATE SET NULL y ON DELETE SET NULL, el comportamiento sería:
 - Si se hubiera creado la tabla del ejemplo anterior de la siguiente manera:

```
CREATE TABLE pagos_pendientes(  
    dni varchar(15),  
    importe double,  
    FOREIGN KEY(dni) REFERENCES clientes(dni)  
    ON DELETE SET NULL  
    ON UPDATE SET NULL  
) engine=innodb;
```

7. BORRADO Y MODIFICACIÓN DE REGISTROS CON RELACIONES

- EJEMPLO SET NULL
 - DELETE FROM clientes WHERE dni='5555672L'; (Se borra el cliente...)

```
mysql> DELETE FROM clientes WHERE dni='5555672L';  
Query OK, 1 row affected (0,00 sec)
```

- Se verifica que han sustituido los valores por NULL

```
mysql> SELECT * FROM pagos_pendientes;  
+-----+-----+  
| dni   | importe |  
+-----+-----+  
| NULL  | 500     |  
| NULL  | 234.5   |  
+-----+-----+
```

7. BORRADO Y MODIFICACIÓN DE REGISTROS CON RELACIONES

- EJEMPLO SET NULL

- Si en lugar de borrar el cliente, se hubiera cambiado el dni:

- UPDATE clientes SET dni='55555555L' WHERE dni='5555672L';

```
mysql> UPDATE clientes SET dni='55555555L'  
-> WHERE dni='5555672L';  
Query OK, 1 row affected (0,02 sec)  
Rows matched: 1  Changed: 1  Warnings: 0
```

- Se verifica que han sustituido los valores por NULL

```
mysql> SELECT * FROM pagos_pendientes;  
+-----+-----+  
| dni | importe |  
+-----+-----+  
| NULL | 500 |  
| NULL | 234.5 |  
+-----+-----+
```

7. BORRADO Y MODIFICACIÓN DE REGISTROS CON RELACIONES

- EJEMPLO SET NULL
 - En este ejemplo vemos cómo se crea automáticamente un índice gestor para la clave foránea.

```
CREATE TABLE clientes (  
  dni varchar(15) PRIMARY KEY,  
  nombre varchar(50),  
  dirección varchar(50)  
) engine=innodb;  
CREATE TABLE pagos_pendientes(  
  dni varchar(15),  
  importe double,  
  FOREIGN KEY(dni) REFERENCES clientes(dni)  
  ON DELETE RESTRICT  
  ON UPDATE RESTRICT  
) engine=innodb;  
#un cliente y dos pagos pendientes  
INSERT INTO clientes  
VALUES ('5555672L', 'Pepe Cifuentes', 'C/Los almendros,23');  
INSERT INTO pagos_pendientes VALUES ('5555672L', 500);  
INSERT INTO pagos_pendientes VALUES ('5555672L', 234.5);
```

```
| pagos_pendientes | CREATE TABLE `pagos_pendientes` (  
  `dni` varchar(15) DEFAULT NULL,  
  `importe` double DEFAULT NULL,  
  KEY `dni` (`dni`),  
  CONSTRAINT `pagos_pendientes_ibfk_1` FOREIGN KEY (`dni`)  
  REFERENCES `clientes` (`dni`)  
) ENGINE=InnoDB DEFAULT CHARSET=latin1 |
```

7. BORRADO Y MODIFICACIÓN DE REGISTROS CON RELACIONES

- UPDATE CASCADE || SET NULL con AUTORREFERENCIAS
 - Si tenemos una tabla con una clave foránea a sí misma (como la tabla empleados con codJefe).
 - Podemos establecer las opciones ON UPDATE CASCADE o ON UPDATE SET NULL.
 - PostgreSQL permite claves foráneas autorreferenciales con ON UPDATE/ON DELETE (CASCADE, SET NULL, RESTRICT/NO ACTION). En modelos con ciclos complejos hay que diseñar bien para evitar dependencias circulares, pero la acción autorreferencial existe. Esto es para evitar bucles infinitos resultantes de actualizaciones en cascada.
-

8. ACCESO A LA INFORMACIÓN

- **Cualquier SGBD debe proporcionar mecanismos de seguridad en el acceso a la información de una base de datos.**
- Estos mecanismos autorizarán a un usuario a utilizar elementos concretos de una base de datos.
 - Las vistas
 - Los Usuarios
 - Los Privilegios

9. VISTAS

- Recordemos que una vista es una tabla virtual (sin contenido) creada a partir de consultas `SELECT` a otras tablas.
 - La diferencia con una consulta ejecutada directamente es que, mientras cada **sentencia SQL** enviada al **SGBD** tiene que pasar por un **proceso de compilación**, la vista es una consulta cuya definición ha sido almacenada previamente y que ya ha sido compilada, siendo, por tanto el tiempo de ejecución bastante menor.
-

9. VISTAS

- También tiene una implicación importante en el hecho de que un usuario podría no tener acceso a la información de varias tablas y, sin embargo, sí tener acceso a la vista que consulta esas tablas, proporcionando de esta manera un acceso controlado solo a determinadas filas y columnas de esa tabla.

```
1 CREATE VIEW test.v AS SELECT * FROM t;
```

10. USUARIOS – ROLES Y PRIVILEGIOS

- A la hora de conectarnos a la base de datos es necesario que utilicemos un modo de acceso, de manera que queden descritos los permisos de que dispondremos durante nuestra conexión.
- En función del nombre de usuario tendremos unos permisos u otros.

10. USUARIOS – ROLES Y PRIVILEGIOS

- Un usuario es un conjunto de permisos que se aplican a una conexión de base de datos.
 - Tiene además otras funciones como son:
 - Ser el propietario de ciertos objetos (tablas, vistas, etc.).
 - Realiza las copias de seguridad.
 - Define una cuota de almacenamiento.
 - Define el tablespace por defecto para los objetos de un usuario
-

10. USUARIOS – ROLES Y PRIVILEGIOS

- Tablespaces
 - Ver los tablespaces -> `cd /var/lib/mysql`
 - Crear tablespace:
 - `CREATE TABLESPACE ts1 ADD DATAFILE '/my/tablespace/directory/ts1.ibd' Engine=InnoDB`
 - **IMPORTANTE:** el directorio debe estar configurado (InnoDB tiene que saber que existe dicho directorio, por defecto no lo sabe). En caso de no establecer la ruta se encontrará en `/var/lib/mysql`.
 - Eliminarlo tablespace:
 - `DROP TABLESPACE ts1;`
-

10. USUARIOS – ROLES Y PRIVILEGIOS

- Pero no todos los usuarios deberían poder hacer lo mismo cuando acceden a la base de datos.
- Por ejemplo, un administrador debería tener más privilegios que un usuario que quiere realizar una simple consulta
- **¿Qué es un privilegio?**

10. USUARIOS – ROLES Y PRIVILEGIOS

- Privilegios
 - Un privilegio es un permiso dado a un usuario para que realice ciertas operaciones, que pueden ser de dos tipos:
 - **De sistema:** necesitará el permiso de sistema correspondiente.
 - **Sobre objeto:** necesitará el permiso sobre el objeto en cuestión.
 - **¿Y no sería interesante poder agrupar esos permisos para darlos juntos?**
 - Para eso tenemos el rol.
-

10. USUARIOS – ROLES Y PRIVILEGIOS

- Rol
 - **Un rol de base de datos no es más que una agrupación de permisos de sistema y de objeto.**
 - EJEMPLO DE USO: Podemos tener a un grupo determinado de usuarios que tengan permiso para consultar los datos de una tabla concreta y no tener permiso para actualizarlos.
 - **Luego un rol permite asignar un grupo de permisos a un usuario.**
 - De este modo, si asignamos un rol con 5 permisos a 200 usuarios y luego queremos añadir un permiso nuevo al rol, no tendremos que ir añadiendo este nuevo permiso a los 200 usuarios, ya que el rol se encarga de propagar automáticamente.
-

10. USUARIOS – ROLES Y PRIVILEGIOS

- ¿Cómo creo un usuario?

10. USUARIOS – ROLES Y PRIVILEGIOS

- Para **crear cuentas de usuario** que permitan a los usuarios acceder a ciertos objetos con un nivel determinado de privilegios hay que hacer uso del comando **CREATE USER**.
 - `CREATE USER nuevo_usuario IDENTIFIED BY 'password' [opciones];`
 - Mediante la orden anterior, podremos crear de forma sencilla un usuario que permite la autenticación en el SGBD con la password identificada mediante la opción “IDENTIFIED BY”.
-

10. USUARIOS – ROLES Y PRIVILEGIOS

- Se pueden especificar muchas más opciones y parámetros en la creación de usuarios (ver enlace a documentación oficial).

```
1 CREATE USER 'jeffrey'@'localhost' IDENTIFIED BY 'password';
```

10. USUARIOS – ROLES Y PRIVILEGIOS

```
CREATE USER [IF NOT EXISTS]
  user [auth_option] [, user [auth_option]] ...
  DEFAULT ROLE role [, role ] ...
  [REQUIRE {NONE | tls_option [[AND] tls_option] ...}]
  [WITH resource_option [resource_option] ...]
  [password_option | lock_option] ...
  [COMMENT 'comment_string' | ATTRIBUTE 'json_object']

user:
  (see Section 6.2.4, "Specifying Account Names")

auth_option: {
  IDENTIFIED BY 'auth_string' [AND 2fa_auth_option]
| IDENTIFIED BY RANDOM PASSWORD [AND 2fa_auth_option]
| IDENTIFIED WITH auth_plugin [AND 2fa_auth_option]
| IDENTIFIED WITH auth_plugin BY 'auth_string' [AND 2fa_auth_option]
| IDENTIFIED WITH auth_plugin BY RANDOM PASSWORD [AND 2fa_auth_option]
| IDENTIFIED WITH auth_plugin AS 'auth_string' [AND 2fa_auth_option]
| IDENTIFIED WITH auth_plugin [initial_auth_option]
}

2fa_auth_option: {
  IDENTIFIED BY 'auth_string' [AND 3fa_auth_option]
| IDENTIFIED BY RANDOM PASSWORD [AND 3fa_auth_option]
| IDENTIFIED WITH auth_plugin [AND 3fa_auth_option]
| IDENTIFIED WITH auth_plugin BY 'auth_string' [AND 3fa_auth_option]
| IDENTIFIED WITH auth_plugin BY RANDOM PASSWORD [AND 3fa_auth_option]
| IDENTIFIED WITH auth_plugin AS 'auth_string' [AND 3fa_auth_option]
}

3fa_auth_option: {
  IDENTIFIED BY 'auth_string'
| IDENTIFIED BY RANDOM PASSWORD
| IDENTIFIED WITH auth_plugin
| IDENTIFIED WITH auth_plugin BY 'auth_string'
| IDENTIFIED WITH auth_plugin BY RANDOM PASSWORD
| IDENTIFIED WITH auth_plugin AS 'auth_string'
}
```

```
initial_auth_option: {
  INITIAL AUTHENTICATION IDENTIFIED BY {RANDOM PASSWORD | 'auth_string'}
| INITIAL AUTHENTICATION IDENTIFIED WITH auth_plugin AS 'auth_string'
}

tls_option: {
  SSL
| X509
| CIPHER 'cipher'
| ISSUER 'issuer'
| SUBJECT 'subject'
}

resource_option: {
  MAX_QUERIES_PER_HOUR count
| MAX_UPDATES_PER_HOUR count
| MAX_CONNECTIONS_PER_HOUR count
| MAX_USER_CONNECTIONS count
}

password_option: {
  PASSWORD EXPIRE [DEFAULT | NEVER | INTERVAL N DAY]
| PASSWORD HISTORY {DEFAULT | N}
| PASSWORD REUSE INTERVAL {DEFAULT | N DAY}
| PASSWORD REQUIRE CURRENT [DEFAULT | OPTIONAL]
| FAILED_LOGIN_ATTEMPTS N
| PASSWORD_LOCK_TIME {N | UNBOUNDED}
}

lock_option: {
  ACCOUNT LOCK
| ACCOUNT UNLOCK
}
```

10. USUARIOS – ROLES Y PRIVILEGIOS

- Observamos que podemos especificar muchas opciones adicionales en la creación de usuarios.
- Aspectos como el tipo de autenticación, cifrados en las conexiones o establecer límites de recursos en el servidor.

10. USUARIOS – ROLES Y PRIVILEGIOS

- Especificar nombre de cuenta en MySQL
 - El nombre de usuario y el nombre del host van entre comillas cada uno.
 - Aunque en algunos casos no es necesario usar las comillas las pondremos siempre.
 - Es correcto 'me'@'localhost'
 - No es correcto 'me@localhost'; - Que sería equivalente a 'me@localhost'@'%'

10. USUARIOS – ROLES Y PRIVILEGIOS

- Especificar nombre de cuenta en MySQL
 - Los comodines % y _ están permitidos en el hostname o direcciones IP
 - Estos comodines tienen el mismo comportamiento que con el operador LIKE
 - Ejemplo: '198.51.100.%' incluye cualquier host en la red de clase C 198.51.100

10. USUARIOS – ROLES Y PRIVILEGIOS

- Borrar usuarios – DROP
 - Para borrar los usuarios, usamos la sentencia:

```
1 DROP USER [IF EXISTS] user [, user] ...
```

- Ejemplo

```
1 DROP USER 'jeffrey'@'localhost';
```

10. USUARIOS – ROLES Y PRIVILEGIOS

- Modificar usuarios – ALTER
 - Para modificar usuarios se utiliza el comando ALTER USER
 - Este comando ALTER USER usa las mismas opciones que CREATE USER.

10. USUARIOS – ROLES Y PRIVILEGIOS

- EJEMPLO 1
 - Cambia la contraseña de una cuenta y hazla caducar.
 - Como resultado, el usuario debe conectarse con la contraseña nombrada y debe elegir uno nuevo en la próxima conexión.

```
1 ALTER USER 'jeffrey'@'localhost'  
2 IDENTIFIED BY 'new_password' PASSWORD EXPIRE;
```

10. USUARIOS – ROLES Y PRIVILEGIOS

- EJEMPLO 2
 - Bloquear o desbloquear una cuenta.

10. USUARIOS – ROLES Y PRIVILEGIOS

- EJEMPLO 2
 - Bloquear o desbloquear una cuenta.

```
1 ALTER USER 'jeffrey'@'localhost' ACCOUNT LOCK;  
2 ALTER USER 'jeffrey'@'localhost' ACCOUNT UNLOCK;
```

10. USUARIOS – ROLES Y PRIVILEGIOS

- EJEMPLO 3
 - Cambia la contraseña del usuario actual sin nombrar su user name.

10. USUARIOS – ROLES Y PRIVILEGIOS

- EJEMPLO 3
 - Cambia la contraseña del usuario actual sin nombrar su user name.

```
1 ALTER USER USER() IDENTIFIED BY 'auth_string';
```

10. USUARIOS – ROLES Y PRIVILEGIOS

- Rename – Usuarios
 - Otros comandos interesantes que nos sirve para renombrar un usuario.

```
1 RENAME USER old_user TO new_user  
2 [, old_user TO new_user] ...
```

- Para usar RENAME USER debes tener privilegios.

```
1 RENAME USER 'jeffrey'@'localhost' TO 'jeff'@'127.0.0.1';
```

10. USUARIOS – ROLES Y PRIVILEGIOS

- **Todas estas sentencias para crear, modificar, eliminar o renombrar usuarios forman parte de las sentencias DDL (en SQL)**

11. LENGUAJE DE CONTROL DE DATOS DCL

- Usuarios
 - **Ya hemos visto que necesitamos una cuenta de usuario para acceder a los datos de una base de datos.**
 - Las claves de acceso se establecen cuando se crea el usuario y pueden ser modificados por el Administrador o por el propietario de dicha clave.
 - CURIOSIDAD: La Base de Datos almacena encriptadas las claves en una tabla del diccionario llamada DBA_USERS.
 - Podemos ver los usuarios desde terminal `>mysql>select ...from user;`
 - También desde terminal `>information_schema> select * from user_attributes;` (existe `user_privileges` para ver los privilegios)
-

11. LENGUAJE DE CONTROL DE DATOS DCL

Sentencia	Descripción
GRANT	Permite dar permisos a uno o varios usuarios o roles para realizar tareas determinadas
REVOKE	Permite eliminar permisos que previamente se han concedido con GRANT

11. LENGUAJE DE CONTROL DE DATOS DCL

- GRANT

- SINTAXIS

```
1 GRANT
2   priv_type [(column_list)]
3   [, priv_type [(column_list)]] ...
4   ON [object_type] priv_level
5   TO user [auth_option] [, user [auth_option]] ...
6   [REQUIRE {NONE | tls_option [[AND] tls_option] ...}]
7   [WITH {GRANT OPTION | resource_option} ...]
```

- **priv_type**: establece la clase de permiso que se otorga.
 - En el siguiente ejemplo el comando GRANT da privilegios globales de SELECT e INSERT

```
1 GRANT SELECT, INSERT ON *.* TO u1;
```

11. LENGUAJE DE CONTROL DE DATOS DCL

- GRANT – priv_type

Privilege	Meaning and Grantable Levels
<u>ALL [PRIVILEGES]</u> ←	Grant all privileges at specified access level except <u>GRANT OPTION</u> and <u>PROXY</u> .
<u>ALTER</u> ←	Enable use of <u>ALTER TABLE</u> . Levels: Global, database, table.
<u>ALTER ROUTINE</u>	Enable stored routines to be altered or dropped. Levels: Global, database, routine.
<u>CREATE</u> ←	Enable database and table creation. Levels: Global, database, table.
<u>CREATE ROUTINE</u>	Enable stored routine creation. Levels: Global, database.
<u>CREATE TABLESPACE</u>	Enable tablespaces and log file groups to be created, altered, or dropped. Level: Global.
<u>CREATE TEMPORARY TABLES</u>	Enable use of <u>CREATE TEMPORARY TABLE</u> . Levels: Global, database.
<u>CREATE USER</u> ←	Enable use of <u>CREATE USER</u> , <u>DROP USER</u> , <u>RENAME USER</u> , and <u>REVOKE ALL PRIVILEGES</u> . Level: Global.
<u>CREATE VIEW</u>	Enable views to be created or altered. Levels: Global, database, table.
<u>DELETE</u> ←	Enable use of <u>DELETE</u> . Level: Global, database, table.
<u>DROP</u> ←	Enable databases, tables, and views to be dropped. Levels: Global, database, table.
<u>EVENT</u>	Enable use of events for the Event Scheduler. Levels: Global, database.
<u>EXECUTE</u>	Enable the user to execute stored routines. Levels: Global, database, routine.
<u>FILE</u>	Enable the user to cause the server to read or write files. Level: Global.
<u>GRANT OPTION</u> ←	Enable privileges to be granted to or removed from other accounts. Levels: Global, database, table, routine, proxy.
<u>INDEX</u> ←	Enable indexes to be created or dropped. Levels: Global, database, table.

11. LENGUAJE DE CONTROL DE DATOS DCL

- GRANT – priv_type

<u>GRANT OPTION</u> ←	Enable privileges to be granted to or removed from other accounts. Levels: Global, database, table, routine, proxy.
<u>INDEX</u> ←	Enable indexes to be created or dropped. Levels: Global, database, table.
<u>INSERT</u> ←	Enable use of <u>INSERT</u> . Levels: Global, database, table, column.
<u>LOCK TABLES</u>	Enable use of <u>LOCK TABLES</u> on tables for which you have the <u>SELECT</u> privilege. Levels: Global, database.
<u>PROCESS</u>	Enable the user to see all processes with <u>SHOW PROCESSLIST</u> . Level: Global.
<u>PROXY</u>	Enable user proxying. Level: From user to user.
<u>REFERENCES</u>	Enable foreign key creation. Levels: Global, database, table, column.
<u>RELOAD</u>	Enable use of <u>FLUSH</u> operations. Level: Global.
<u>REPLICATION CLIENT</u>	Enable the user to ask where master or slave servers are. Level: Global.
<u>REPLICATION SLAVE</u>	Enable replication slaves to read binary log events from the master. Level: Global.
<u>SELECT</u> ←	Enable use of <u>SELECT</u> . Levels: Global, database, table, column.
<u>SHOW DATABASES</u> ←	Enable <u>SHOW DATABASES</u> to show all databases. Level: Global.
<u>SHOW VIEW</u>	Enable use of <u>SHOW CREATE VIEW</u> . Levels: Global, database, table.
<u>SHUTDOWN</u>	Enable use of mysqladmin shutdown . Level: Global.
<u>SUPER</u>	Enable use of other administrative operations such as <u>CHANGE MASTER TO</u> , <u>KILL</u> , <u>PURGE BINARY LOGS</u> , <u>SET GLOBAL</u> , and mysqladmin debug command. Level: Global.
<u>TRIGGER</u>	Enable trigger operations. Levels: Global, database, table.
<u>UPDATE</u> ←	Enable use of <u>UPDATE</u> . Levels: Global, database, table, column.
<u>USAGE</u>	Synonym for “no privileges”

11. LENGUAJE DE CONTROL DE DATOS DCL

- Los Privilegios
 - Los privilegios se pueden englobar en las siguientes categorías:
 - Privilegios Globales: administrar bases de datos y cuentas de usuario, privilegios sobre las tablas,...
 - Privilegios de bases de datos: permiten crear tablas y funciones, procedimientos, en una BD,...
 - Privilegios de tabla: seleccionar y cambiar datos en una tabla,...
-

11. LENGUAJE DE CONTROL DE DATOS DCL

- Los Privilegios
 - Los privilegios se pueden englobar en las siguientes categorías:
 - Privilegios de columna: establecen que usuarios pueden cambiar o seleccionar columnas concretas de una tabla,...
 - Privilegios de función: ejecutar una función determinada, dar permisos, modificar funciones,...
 - Privilegios de procedimiento: ejecutar un procedimiento determinado usando la sentencia CALL, dar permisos sobre un procedimiento,...
-

11. LENGUAJE DE CONTROL DE DATOS DCL

- Los Privilegios – `priv_level`

- El siguiente elemento a configurar dentro de la sentencia es `ON`, y define en donde se aplica el permiso:

```
13 object_type: {  
14     TABLE  
15     | FUNCTION  
16     | PROCEDURE  
17 }  
18  
19 priv_level: {  
20     *  
21     | *.*  
22     | db_name.*  
23     | db_name.tbl_name  
24     | tbl_name  
25     | db_name.routine_name  
26 }
```

EXPRESIÓN	Se aplica permiso a
Tbl_name	A la tabla nombre_tabla
*	Todas las tablas de la DB que se esté usando
.	Todas las tablas de todas las BBDD
Db_name.*	Todas las tablas de la BD db_name
Db_name.tbl_name	Sólo la tabla tbl_name de la BD db_name

11. LENGUAJE DE CONTROL DE DATOS DCL

- Los Privilegios – resource_option
 - **TO usuario**: indica el usuario al que se le quiere otorgar el permiso.

11. LENGUAJE DE CONTROL DE DATOS DCL

- Los Privilegios – resource_option
 - **WITH** establece ciertas opciones adicionales:

OPCIÓN	FINALIDAD
GRANT OPTION	Permite conceder a otros usuarios los permisos que tiene el usuario
MAX_QUERIES_PER_HOUR count	Permite restringir el número de consultas por hora que puede realizar un usuario.
MAX_UPDATES_PER_HOUR count	Permite restringir el número de modificaciones por hora que puede realizar un usuario
MAX_CONNECTIONS_PER_HOUR count	Permite restringir las conexiones (logins) por hora que realiza un usuario
MAX_USER_CONNECTIONS count	Permite limitar el nº de conexiones simultáneas de un usuario

- Nota: si asignamos 0 al valor de count, significa ilimitado.
-

11. LENGUAJE DE CONTROL DE DATOS DCL

- REVOKE

- De la misma forma que otorgamos permisos, en muchas ocasiones debemos desasignarlos por diferentes motivos.

- Su sintaxis es muy parecida al GRANT:

```
1 REVOKE
2   priv_type [(column_list)]
3   [, priv_type [(column_list)]] ..
4   ON [object_type] priv_level
5   FROM user [, user] ...
6
7 REVOKE ALL [PRIVILEGES], GRANT OPTION
8   FROM user [, user] ...
9
10 REVOKE PROXY ON user
11   FROM user [, user] ...
```

```
REVOKE SHOW DATABASES ON *.* FROM 'simplon'@'localhost';
```

¡CUIDADO!

No hace falta poner ON cuando vamos a quitar todos los privilegios y además MySQL entiende que si le vas a quitar todo es TODO hasta el privilegio de dar permisos.

11. LENGUAJE DE CONTROL DE DATOS DCL

- REVOKE
 - EJEMPLOS
 - Quita el permiso de SELECT en la tabla futbol.equipos
 - Elimina el permiso ALL PRIVILEGES de todas las tablas
 - Quita los permisos de SELECT e INSERT de todas las tablas de jardinería
-

11. LENGUAJE DE CONTROL DE DATOS DCL

- REVOKE

- EJEMPLOS

- Quita el permiso de SELECT en la tabla futbol.equipo

```
REVOKE SELECT ON futbol.equipo FROM 'simplon'@'localhost';
```

- Elimina el permiso ALL PRIVILEGES de todas las tablas

```
REVOKE ALL PRIVILEGES ON *.* FROM 'simplon'@'localhost';
```

- Quita los permisos de SELECT e INSERT de todas las tablas de jardinería

```
REVOKE SELECT, INSERT ON jardineria.* FROM 'simplon'@'localhost';
```

12. PRIVILEGIOS

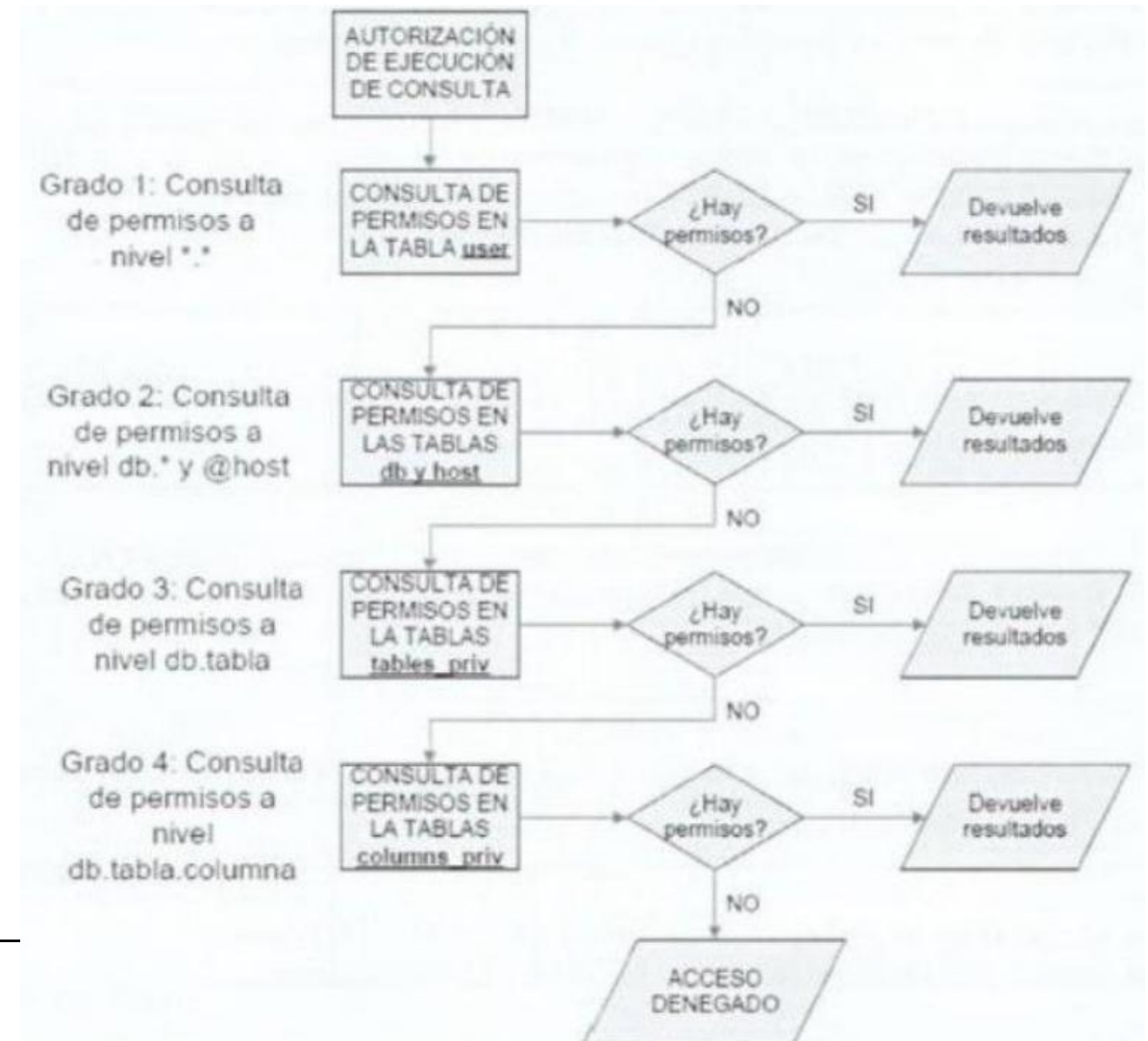
- Tablas
 - **Otra opción de asignar y eliminar permisos** es utilizando las tablas de la BBDD de mysql que se crean junto con la instalación del SGBD.
 - Hay 5 tablas relacionadas con el sistema de permisos:
 - user
 - db
 - host
 - tables_priv
 - columns_priv
-

12. PRIVILEGIOS

- Tablas
 - Viendo la finalidad que tienen estas tablas, es lógico pensar que las modificaciones que se puedan hacer sobre ellas pueden ser peligrosas si no se sabe lo que se está haciendo.
 - **Para usuarios no avanzados, es mejor realizar las modificaciones de permisos con los comandos que me proporciona el SGBD.**

12. PRIVILEGIOS

- Tablas
 - En la ilustración se puede ver como MySQL procesa una consulta.
 - Al recibir la instrucción SQL, se comprueba en ese orden el acceso a los objetos.
 - Si ninguno de los niveles permite el acceso, se deniega la consulta.



12. PRIVILEGIOS

- Algunas instrucciones que nos conviene conocer para la gestión de usuarios y permisos:
 - **FLUSH PRIVILEGES**
 - El SGBD mantiene una copia en memoria de las tablas GRANT.
 - Éste comando fuerza a la recarga de esta caché, leyéndola directamente de disco. Suele usarse cuando utilizamos INSERT, UPDATE o DELETE para modificar las tablas de privilegios directamente.
 - Con las sentencias propias de GRANT, REVOKE,... se cargan las tablas en memoria inmediatamente después de su ejecución.
 - **TABLA mysql.user**
 - Contiene la información de los usuarios en el SGBD.
 - Permite consultar las columnas user, password y host además de los privilegios
-

12. PRIVILEGIOS

- Algunas instrucciones que nos conviene conocer para la gestión de usuarios y permisos:
 - **SHOW GRANTS**
 - Devuelve los permisos que tiene el usuario actual.
 - **SHOW GRANTS FOR USER@HOST**
 - Devuelve los permisos de un usuario determinado
 - **SELECT USER() ó SELECT CURRENT_USER()**
 - Devuelve el usuario que está conectado actualmente en la sesión
-

13. ROLES

```
CREATE ROLE [IF NOT EXISTS] role [, role ] ...
```

- Creamos un rol
 - CREATE ROLE 'developer';
 - Asignamos los permisos al rol que hemos creado
 - GRANT CREATE, SELECT, INSERT, UPDATE, DELETE on db_empresa.* to 'developer';
 - Mostramos los permisos del ROL
 - SHOW GRANTS FOR 'developer';
-

13. ROLES

```
CREATE ROLE [IF NOT EXISTS] role [, role ] ...
```

- Asignamos los permisos del rol al usuario
 - GRANT developer to 'paco'@'localhost';
- Al mostrar los permisos del usuario, mostrará el rol que se le asignó
 - SHOW GRANTS FOR 'paco'@'localhost';

13. ROLES

```
CREATE ROLE [IF NOT EXISTS] role [, role ] ...
```

- No es posible quitarle un privilegio al usuario cuando se le ha asignado un rol
 - REVOKE CREATE FROM 'paco'@'localhost'; - Esto se debe a que el privilegio está asignado al rol no al user.
 - Lo que sí es posible es eliminar el rol al usuario (con lo que perderá todos los privilegios)
 - REVOKE 'developer' FROM 'paco'@'localhost';
 - También se pueden modificar los privilegios asignados a un rol, pero afectará a todos los usuarios que se les haya asignado ese rol.
 - REVOKE create, insert, update, delete, select on db_empresa.* from 'developer';
 - Se puede eliminar un rol y automáticamente se eliminará de los usuarios que lo tengan asignado.
 - DROP ROLE 'developer';
-

13. ROLES

- Por defecto, no se activan automáticamente los permisos asignados a un rol de un usuario.
 - Hay dos formas de utilizar los permisos de un rol, desde root (o admin) ejecutamos:
 - SET ROLE nombreRol **(Debe activarlo el usuario cuando quiere esos permisos)**
 - SET ROLE usuarioTriste
 - SET DEFAULT ROLE nombreRol **(No debe hacer nada el usuario)**
 - SET default ROLE usuarioTriste to user3@localhost;
 - SET PERSIST activate_all_roles_on_login = ON;
 - Con este se activan todos los roles de todos los usuarios
 - Debes tener los privilegios de SYSTEM_VARIABLES_ADMIN y ROLE_ADIMIN
 - CREATE USER 'user4'@'localhost' identified by '1234' default role usuarioTriste;
-

13. ROLES

- Curiosidad
 - Si lo que quieres es dar permisos para realizar copias de seguridad basta con:
 - `GRANT SELECT,SHOW DATABASES,LOCK TABLES On *.* To 'paco'@'localhost';`
 - Con estos tres permisos estamos permitiendo que el usuario: acceda y vea las BD, bloquee las tablas y lea la información de las mismas(select).
 - Existen diferentes tipos de copias de seguridad y existen diferentes sentencias en función del motor de almacenamiento de las tablas. Para tablas MyISAM, estos permisos son suficientes (mysqldump). Si tuviésemos tablas InnoDB necesitamos el permiso RELOAD (mysqlhotcopy). `GRANT SELECT, SHOW DATABASES, LOCK TABLES, RELOAD On *.* To 'paco'@'localhost';`
 - También existen los privilegios dinámicos, algunos son específicos para realizar copias de seguridad:
<https://dev.mysql.com/doc/refman/8.0/en/privileges-provided.html>
-

13. ROLES

- Curiosidad
 - Crear tablespace
 - `CREATE TABLESPACE pacoTablespace ADD DATAFILE 'pacoTablespace.ibd';`
 - Crear usuario y dar privilegios
 - `CREATE USER 'paco'@'localhost' IDENTIFIED BY '1234';`
 - `GRANT ALL PRIVILEGES ON *.* TO 'paco'@'localhost';`
 - Crear tabla con tablespace asignado
 - `CREATE TABLE tablaPaco(id int primary key, nombre char(50)) tablespace=pacoTablespace;`
-

14. MOTORES DE ALMACENAMIENTO

- COPIAS DE SEGURIDAD
 - InnoDB. En la teoría de la UT03 estudiamos lo que era un motor de almacenamiento en un SGBD y vimos algunos casos en los que se usan con frecuencia.
 - Recordatorio: `SHOW ENGINES;` /* Muestra todos los motores de almacenamiento soportados*/
 - En nuestro caso particular, como usamos integridad referencial en las tablas a través de claves foráneas, necesitamos hacer uso del motor InnoDB.
-

15. TRUNCAR TABLAS

- Para vaciar una tabla completamente podemos usar la sentencia TRUNCATE, cuya sintaxis es:
 - `TRUNCATE TABLE NombreTabla;`

15. TRUNCAR TABLAS

- Aunque TRUNCATE TABLE es similar a DELETE, se diferencian en:
 - **La operación de truncar elimina la tabla y la vuelve a crear** (lenguaje de definición de datos – LDD) lo que lo hace más rápido que eliminar filas una a una particularmente en tablas grandes.
 - **Cualquier valor auto_increment será reseteado al valor inicial.**
 - TRUNCATE TABLE no invoca los triggers al eliminar (delete).
 - TRUNCATE TABLE realiza commit de forma implícita y no puede deshacer los cambios (DDL).
-

15. TRUNCAR TABLAS

- A continuación, se muestra el resultado de aplicar delete y truncate:

```
CREATE table tablaEjemplo (  
    id int auto_increment primary key,  
    idEmple int not null,  
    FOREIGN KEY (idEmple) references emple(codEmple)  
    on delete restrict);
```

id	idEmple
1	200
2	201

```
insert into tablaEjemplo values(default, 200), (default,201);
```

```
106 • delete from tablaEjemplo;  
107 • insert into tablaEjemplo values(default, 200), (default,201);  
108 • select * from tablaEjemplo;
```

Result Grid			Filter Rows:	Edit:	Export/Import:
#	id	idEmple			
1	3	200			
2	4	201			

15. TRUNCAR TABLAS

- A continuación, se muestra el resultado de aplicar delete y truncate:

```
CREATE table tablaEjemplo (  
    id int auto_increment primary key,  
    idEmple int not null,  
    FOREIGN KEY (idEmple) references emple(codEmple)  
    on delete restrict);
```

id	idEmple
1	200
2	201

```
insert into tablaEjemplo values(default, 200), (default,201);
```

```
106 • truncate tablaEjemplo;  
107 • insert into tablaEjemplo values(default, 200), (default,201);  
108 • select * from tablaEjemplo;
```

Result Grid			Filter Rows:	Edit:	Export/Import:
#	id	idEmple			
1	1	200			
2	2	201			

15. TRUNCAR TABLAS

- De la misma forma que no podíamos eliminar una tabla que tenga una clave ajena, tampoco podremos truncarla, es decir, no podemos truncar la tabla emple si existe la tablaEjemplo

16. TRANSACCIONES

- **Una transacción es un grupo de acciones que hacen transformaciones en las tablas preservando la consistencia de los datos.**
 - Inserción, modificación o eliminación de datos.
- **Es decir, una transacción es un conjunto de sentencias SQL que se tratan como una sola instrucción (atómica).**
 - O se hacen todas las operaciones, o no se hace ninguna.

16. TRANSACCIONES

- No todas las operaciones SQL son transaccionales.
- Sólo son transaccionales las operaciones correspondientes al LMD, es decir, sentencias:
 - SELECT
 - INSERT
 - UPDATE
 - DELETE

16. TRANSACCIONES

- No todos los motores de almacenamiento permiten el uso de transacciones
- En MySQL el único motor que admite las transacciones es InnoDB (motor de almacenamiento por defecto)
 - Recordatorio: SHOW ENGINES; - Muestra todos los motores de almacenamiento soportados

```
mysql> show engines;
```

Engine	Support	Comment	Transactions	XA	Savepoints
CSV	YES	CSV storage engine	NO	NO	NO
MRG_MYISAM	YES	Collection of identical MyISAM tables	NO	NO	NO
InnoDB	DEFAULT	Supports transactions, row-level locking, and foreign keys	YES	YES	YES
BLACKHOLE	YES	/dev/null storage engine (anything you write to it disappears)	NO	NO	NO
PERFORMANCE_SCHEMA	YES	Performance Schema	NO	NO	NO
MEMORY	YES	Hash based, stored in memory, useful for temporary tables	NO	NO	NO
ARCHIVE	YES	Archive storage engine	NO	NO	NO
MyISAM	YES	MyISAM storage engine	NO	NO	NO
FEDERATED	NO	Federated MySQL storage engine	NULL	NULL	NULL

17. COMMIT Y ROLLBACK

- COMMIT

- Sentencia que se utiliza para confirmar una transacción.
- Cuando realizamos COMMIT los cambios se escriben en la BD.



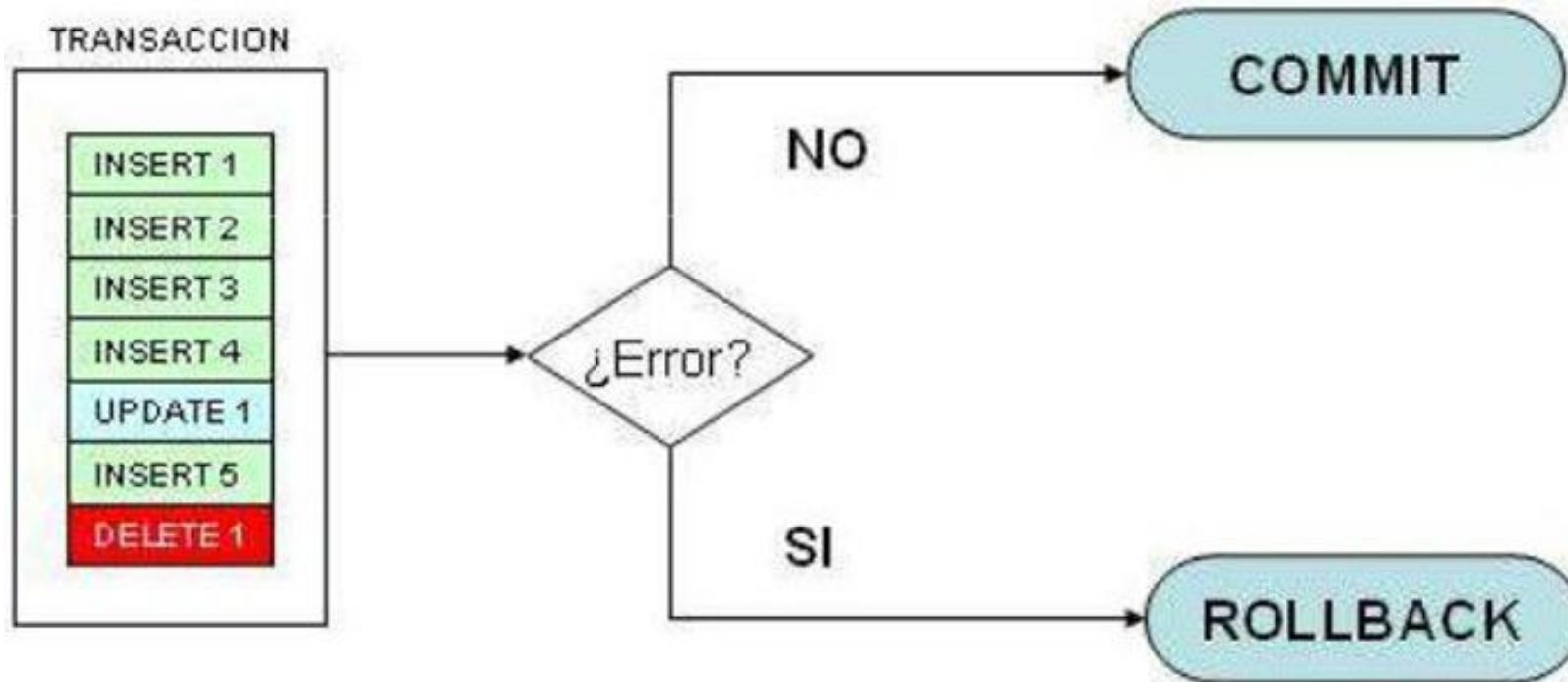
- ROLLBACK

- Sentencia que se utiliza para deshacer una transacción.
- Cuando realizamos ROLLBACK se deshacen todas las modificaciones realizadas por la transacción en la BD, quedando ésta en el mismo estado que antes de iniciarse la transacción.



17. COMMIT Y ROLLBACK

- Flujo de una transacción



17. COMMIT Y ROLLBACK

- EJEMPLO
 - Queremos comprar un videojuego por Internet y vemos que en la web pone que queda sólo 1 en stock.
 - Al añadirlo a la cesta y darle a realizar compra se actualiza el stock a 0.
 - Si al realizar el pago se produce un error en la tarjeta si no hay posibilidad de hacer Rollback el stock se descuadraría
-

17. COMMIT Y ROLLBACK

- AUTOCOMMIT
 - **Generalmente, cuando se conecta con un cliente a un SGBD, por defecto está activado el modo AUTOCOMMIT=ON**
 - Es decir, cada comando SQL que se ejecute, será considerado como una transacción independiente.
 - Se ejecuta el comando COMMIT de forma automática e inmediata después de cada sentencia.
 - UPDATE clientes SET nombre='Néstor' WHERE id=12;
 - INSERT INTO clientes VALUES (15, 'Pedro');

17. COMMIT Y ROLLBACK

- COMMIT
 - **En una transacción los datos modificados no son visibles por el resto de usuarios hasta que se confirme la transacción.**
 - Mientras se procesa una transacción, ningún otro usuario puede realizar cambios sobre los datos afectados por ella.

17. COMMIT Y ROLLBACK

- ROLLBACK
 - La orden ROLLBACK, aborta la transacción o los cambios, volviendo a la situación de las tablas de la BD desde el último COMMIT.
 - Si AUTOCOMMIT (confirmación automática) está activada, de nada servirá hacer un ROLLBACK.
 - Es decir, para que podamos aplicar ROLLBACK, deberemos tener la confirmación automática desactivada.
-

17. COMMIT Y ROLLBACK

- Transacciones
 - **Algunas órdenes SQL llevan implícita la finalización de una transacción.**
 - En general esto ocurre con todas las instrucciones que pertenecen al LDD como crear, modificar y borrar tablas, bases de datos, índices, etc.
 - Si se incluyen en una transacción una instrucción LDD y esta falla, la transacción no podrá deshacerse.

17. COMMIT Y ROLLBACK

- Transacciones
 - MySQL, 3 formas para comenzar una transacción:
 - SET AUTOCOMMIT=0; #OFF
 - START TRANSACTION;
 - BEGIN WORK;
 - La palabra clave WORK es opcional

17. COMMIT Y ROLLBACK

- SET AUTOCOMMIT = 0 (off)
 - Si deshabilitamos AUTOCOMMIT todas las sentencias se ejecutan como si se tratara de una transacción abierta que se termina con ROLLBACK o COMMIT.
 - **Si la conexión se cierra sin confirmar, la transacción se cancela.**

17. COMMIT Y ROLLBACK

- SET AUTOCOMMIT = 1 (on)
 - Aunque cada sentencia se ejecuta automáticamente puedes trabajar con transacciones temporalmente, mediante los siguientes pasos
 1. Empezamos con START TRANSACTION o BEGIN
 2. Sentencias de modificación
 3. Terminamos con ROLLBACK o COMMIT.
 - a. Es como si deshabilitáramos temporalmente la autocomfirmación.
 - b. **Una vez terminada la transacción el modo AUTOCOMMIT vuelve a su estado activado.**

17. COMMIT Y ROLLBACK (RESUMEN)

SET AUTOCOMMIT = 0	A ejecutar esta sentencia, MySQL queda configurado para que las órdenes de modificación que se ejecuten a continuación muestren los datos tal como quedarían pero realmente no se han modificado en la base de datos. La confirmación definitiva se realiza con COMMIT, si se cierra el programa sin confirmar la transacción, ésta no se realiza. Cuando ponemos este estado, dicho tratamiento de transacciones permanece hasta que lo cambiemos.
SET AUTOCOMMIT = 1	El sistema se comporta de forma que una instrucción queda confirmada sobre la base de datos en el momento que se ejecuta. A este modo de confirmación se le denomina modo AUTOCOMMIT.
COMMIT	Cuando se ejecuta esta sentencia el sistema actualiza la base de datos con todos los cambios realizados desde la última vez que se ejecutó esta sentencia. Entre dos ejecuciones de COMMIT podríamos ver el efecto de los cambios pero son cambios virtuales, que no se han producido en la realidad.
ROLLBACK	Cuando tenemos el estado AUTOCOMMIT = 0, ROLLBACK devuelve la base de datos al estado en que se encontraba cuando se realizó el anterior COMMIT o cuando se estableció AUTOCOMMIT a 0. Cuando se produce un fallo de software o de hardware que produce una salida del SGBD accidental, al volver a iniciar el sistema, se produce un ROLLBACK automático y la base de datos vuelve a un estado anterior.

17. COMMIT Y ROLLBACK

- EJEMPLO AUTOCOMMIT = ON
 - Por defecto
 - Se comprueba que por defecto ROLLBACK no funciona

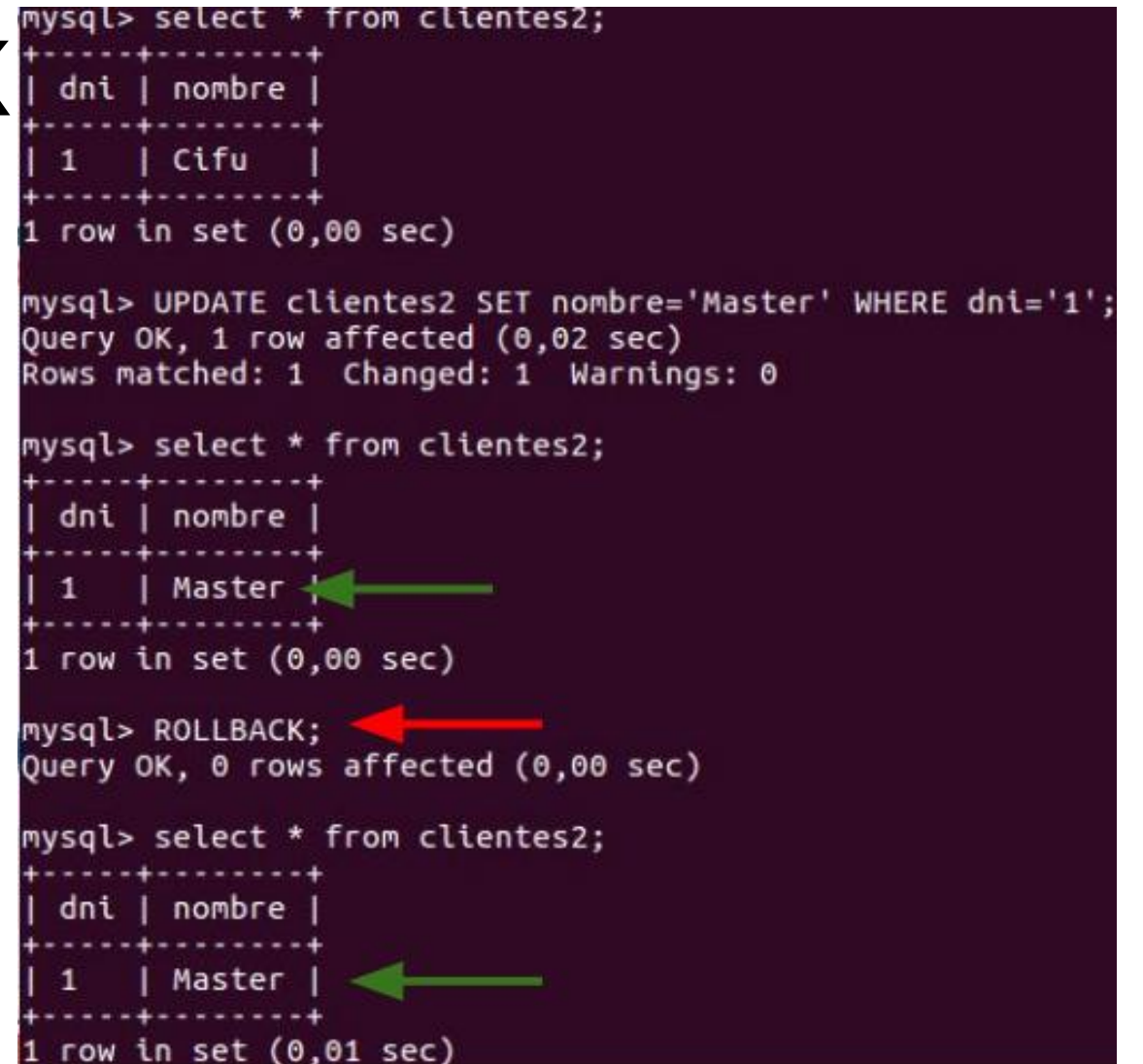
```
mysql> select * from clientes2;
+-----+-----+
| dni | nombre |
+-----+-----+
| 1   | Cifu   |
+-----+-----+
1 row in set (0,00 sec)

mysql> UPDATE clientes2 SET nombre='Master' WHERE dni='1';
Query OK, 1 row affected (0,02 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from clientes2;
+-----+-----+
| dni | nombre |
+-----+-----+
| 1   | Master |
+-----+-----+
1 row in set (0,00 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0,00 sec)

mysql> select * from clientes2;
+-----+-----+
| dni | nombre |
+-----+-----+
| 1   | Master |
+-----+-----+
1 row in set (0,01 sec)
```



17. COMMIT Y ROLLBACK

- EJEMPLO START TRANSACTION
 - ROLLBACK funciona y aborta los cambios.

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0,00 sec)

mysql> UPDATE clientes2 SET nombre='GRATIS' WHERE dni='1';
Query OK, 1 row affected (0,00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from clientes2;
+-----+-----+
| dni | nombre |
+-----+-----+
| 1   | GRATIS | 
+-----+-----+
1 row in set (0,00 sec)

mysql> ROLLBACK; 
Query OK, 0 rows affected (0,01 sec)

mysql> select * from clientes2;
+-----+-----+
| dni | nombre |
+-----+-----+
| 1   | Master | 
+-----+-----+
1 row in set (0,00 sec)
```

17. COMMIT Y ROLLBACK

CONEXIÓN 1

- EJEMPLO START TRANSACTION

```
mysql> BEGIN;
Query OK, 0 rows affected (0,00 sec)

mysql> INSERT INTO clientes2
    -> VALUES ('2', 'Gratis');
Query OK, 1 row affected (0,00 sec)

mysql> select * from clientes2;
+-----+-----+
| dni | nombre |
+-----+-----+
| 1   | Master |
| 2   | Gratis |
+-----+-----+

mysql> COMMIT;
Query OK, 0 rows affected (0,01 sec)
```

CONEXIÓN 2

```
mysql> select * from clientes2;
+-----+-----+
| dni | nombre |
+-----+-----+
| 1   | Master |
+-----+-----+
```

En una transacción los datos modificados no son visibles por el resto de usuarios hasta que se confirme la transacción.

```
mysql> select * from clientes2;
+-----+-----+
| dni | nombre |
+-----+-----+
| 1   | Master |
| 2   | Gratis |
+-----+-----+
```

17. COMMIT Y ROLLBACK

CONEXIÓN 1

CONEXIÓN 2

- EJEMPLO SET
AUTOCOMMIT = 0

```
mysql> SET AUTOCOMMIT=0;
Query OK, 0 rows affected (0,00 sec)

mysql> INSERT INTO clientes2
-> VALUES ('2', 'Gratis');
Query OK, 1 row affected (0,01 sec)

mysql> UPDATE clientes2
-> SET nombre='Cifu' WHERE dni='1';
Query OK, 1 row affected (0,00 sec)
```

```
mysql> select * from clientes2;
+-----+-----+
| dni | nombre |
+-----+-----+
| 1   | Cifu   |
| 2   | Gratis |
+-----+-----+
```

```
mysql> COMMIT;
```

```
mysql> select * from clientes2;
+-----+-----+
| dni | nombre |
+-----+-----+
| 1   | Cifu   |
| 2   | Gratis |
+-----+-----+
```

```
mysql> select * from clientes2;
+-----+-----+
| dni | nombre |
+-----+-----+
| 1   | Master |
+-----+-----+
```

```
mysql> select * from clientes2;
+-----+-----+
| dni | nombre |
+-----+-----+
| 1   | Master |
+-----+-----+
```

```
mysql> select * from clientes2;
+-----+-----+
| dni | nombre |
+-----+-----+
| 1   | Cifu   |
| 2   | Gratis |
+-----+-----+
```

17. COMMIT Y ROLLBACK

CONEXIÓN 1

CONEXIÓN 2

- EJEMPLO SET
AUTOCOMMIT = 0

```
mysql> DELETE FROM clientes2  
-> WHERE dni='2';  
Query OK, 1 row affected (0,00 sec)
```

```
mysql> select * from clientes2;  
+-----+-----+  
| dni | nombre |  
+-----+-----+  
| 1   | Master |  
+-----+-----+
```

```
mysql> select * from clientes2;  
+-----+-----+  
| dni | nombre |  
+-----+-----+  
| 1   | Cifu   |  
| 2   | Gratis |  
+-----+-----+
```

```
mysql> COMMIT;
```

```
mysql> select * from clientes2;  
+-----+-----+  
| dni | nombre |  
+-----+-----+  
| 1   | Master |  
+-----+-----+
```

```
mysql> select * from clientes2;  
+-----+-----+  
| dni | nombre |  
+-----+-----+  
| 1   | Master |  
+-----+-----+
```

17. COMMIT Y ROLLBACK

CONEXIÓN 1

- EJEMPLO SET
AUTOCOMMIT = 0

```
mysql> select * from clientes2;  
+-----+-----+  
| dni | nombre |  
+-----+-----+  
| 1   | Cifu   |  
| 2   | Gratis |  
+-----+-----+
```

Cuando hacemos un SET
AUTOCOMMIT=0; se desactiva para
todas las sesiones, no sólo para
nuestra sesión

```
mysql> select * from clientes2;  
+-----+-----+  
| dni | nombre |  
+-----+-----+  
| 1   | Master |  
+-----+-----+
```

CONEXIÓN 2

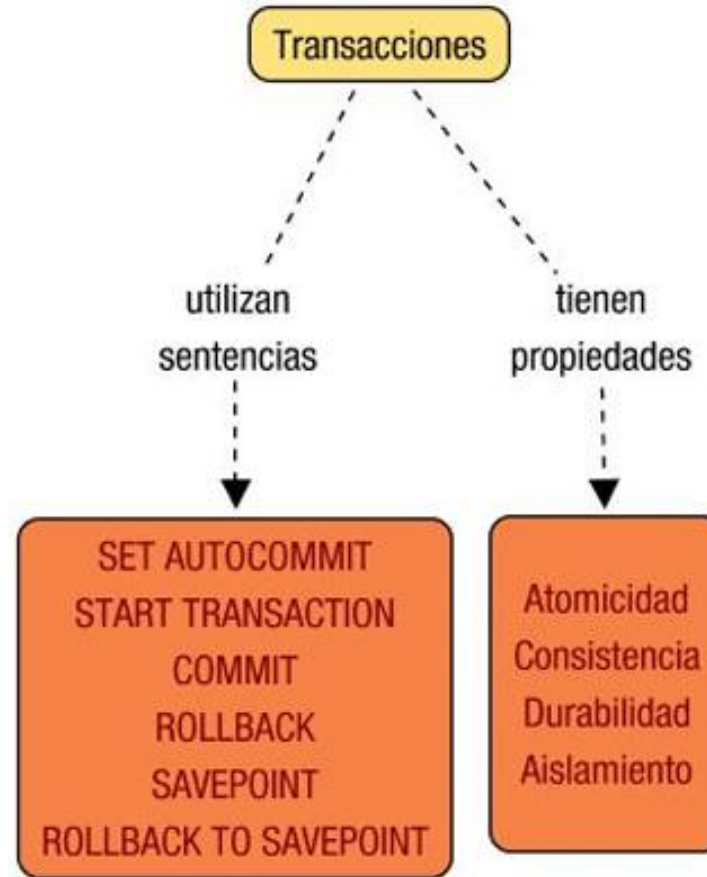
```
mysql> DELETE FROM clientes2  
-> WHERE dni='2';  
Query OK, 1 row affected (0,00 sec)  
  
mysql> select * from clientes2;  
+-----+-----+  
| dni | nombre |  
+-----+-----+  
| 1   | Cifu   |  
+-----+-----+
```

```
mysql> COMMIT;
```

```
mysql> select * from clientes2;  
+-----+-----+  
| dni | nombre |  
+-----+-----+  
| 1   | Master |  
+-----+-----+
```

17. COMMIT Y ROLLBACK

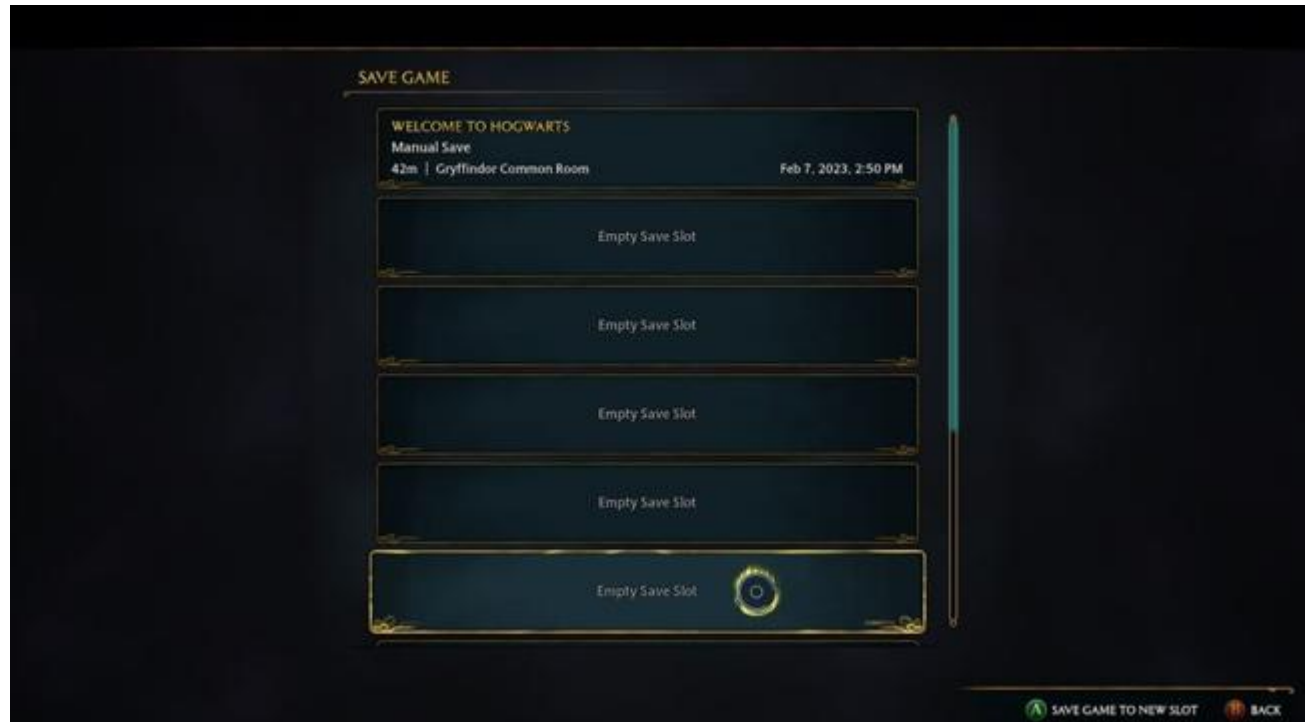
- ESQUEMA



18. SAVEPOINT

```
1 SAVEPOINT identifier
2 ROLLBACK [WORK] TO [SAVEPOINT] identifier
3 RELEASE SAVEPOINT identifier
```

- InnoDB admite y soporta las sentencias SQL SAVEPOINT, ROLLBACK TO SAVEPOINT, RELEASE SAVEPOINT.



18. SAVEPOINT

- SAVEPOINT crea un punto de guardado en una transacción, para ello le establecemos un nombre identificador al punto de guardado.
 - **Si la transacción actual tiene un savepoint con el mismo nombre, el antiguo se elimina y se establece el nuevo.**
 - El ROLLBACK TO SAVEPOINT se ejecuta hasta retroceder al SAVEPOINT indicado sin terminar de ejecutar la transacción.
 - Los SAVEPOINTS que se establecieron en un momento posterior al del punto de guardado nombrado se eliminan.
 - Hay que tener en cuenta que para usar savepoints hay que empezar una transacción antes de definir el primer savepoints.
-

18. SAVEPOINT

- La `RELEASE SAVEPOINT` declaración elimina el punto de guardado nombrado del conjunto de puntos de guardado de la transacción actual.
 - No se produce confirmación ni reversión. Es un error si el punto de guardado no existe.
- Todos los `SAVEPOINT` de la transacción actual se eliminan si se ejecuta un `COMMIT`, o un `ROLLBACK`.

18. SAVEPOINT

- EJEMPLO

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0,00 sec)

mysql>
mysql> INSERT INTO clientes3
    -> VALUES ('1', 'Mequetrefe');
Query OK, 1 row affected (0,00 sec)

mysql>
mysql> SELECT * FROM clientes3;
+-----+-----+
| dni | nombre |
+-----+-----+
| 1   | Mequetrefe |
+-----+-----+
1 row in set (0,01 sec)

mysql>
mysql> SAVEPOINT punto1;
Query OK, 0 rows affected (0,00 sec)

mysql>
mysql> INSERT INTO clientes3
    -> VALUES ('2', 'Sanaca');
Query OK, 1 row affected (0,01 sec)
```

```
mysql> SELECT * FROM clientes3;
+-----+-----+
| dni | nombre |
+-----+-----+
| 1   | Mequetrefe |
| 2   | Sanaca |
+-----+-----+
2 rows in set (0,00 sec)

mysql>
mysql> ROLLBACK TO punto1;
Query OK, 0 rows affected (0,01 sec)

mysql>
mysql> SELECT * FROM clientes3;
+-----+-----+
| dni | nombre |
+-----+-----+
| 1   | Mequetrefe |
+-----+-----+
1 row in set (0,00 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0,00 sec)

mysql> SELECT * FROM clientes3;
Empty set (0,00 sec)
```

19. PROPIEDADES DE LAS TRANSACCIONES

- **Las transacciones presentan una serie de propiedades** que garantizan su fiabilidad. Esas propiedades, **denominadas ACID** (atomicity, consistency, isolation, and durability), son las siguientes:
 - **Atomicidad.** Una transacción es una unidad indivisible, o se ejecutan todas las órdenes de forma adecuada o no se guarda ningún cambio producido sobre los datos.
 - **Consistencia.** Cuando una transacción se ejecuta con éxito la base de datos estará en un estado consistente, y si no se ejecuta con éxito se deshacen las órdenes de la transacción regresando al estado de consistencia anterior.
 - **Durabilidad.** Los cambios que produce una transacción una vez finalizada son permanentes, aunque se produzcan fallos posteriores en el sistema.
-

19. PROPIEDADES DE LAS TRANSACCIONES

- **Las transacciones presentan una serie de propiedades** que garantizan su fiabilidad. Esas propiedades, **denominadas ACID** (atomicity, consistency, isolation, and durability), son las siguientes:
 - **Aislamiento.** Si varias transacciones se realizan de forma concurrente por distintos usuarios, el mismo elemento de datos que está siendo utilizado por una transacción no puede ser utilizado por otra hasta que la anterior termine.
 - Esto se traduce en que los cambios que produce una transacción no sean visibles por las demás hasta que ésta termine.
 - Si no se mantiene el nivel adecuado de aislamiento pueden darse distintos resultados al trabajar con varias transacciones.
-

EJERCICIO 17

```
mysql> select * from clientes;
+-----+-----+-----+
| dni   | nombre   | dirección   |
+-----+-----+-----+
| 5555672L | Pepe Cifuentes | C/Los almendros,23 |
+-----+-----+-----+
```

```
mysql> select * from pagos_pendientes;
+-----+-----+
| dni   | importe |
+-----+-----+
| 5555672L | 250 |
| 5555672L | 500 |
+-----+-----+
```

- **Realizar los siguientes pasos sobre la base de datos clientes_pagos: (Realizar el diseño físico e insertar los datos como aparece en las capturas)**
 1. Iniciar la transacción
 2. Modificar el importe 250 de la tabla pagos a 300
 3. Insertar un nuevo cliente con tu nombre dirección y dni
 4. Insertar un pago pendiente de 500€ al nuevo cliente
 5. Eliminar el importe de 500€ de Pepe Cifuentes.
 6. Verificar los cambios
 7. Confirmar la Transacción
 8. ¿Si da error en alguna sentencia que comando debemos ejecutar?

EJERCICIO 17

```
mysql> select * from clientes;
+-----+-----+-----+
| dni   | nombre      | dirección      |
+-----+-----+-----+
| 5555672L | Pepe Cifuentes | C/Los almendros,23 |
+-----+-----+-----+
```

```
mysql> select * from pagos_pendientes;
+-----+-----+
| dni   | importe |
+-----+-----+
| 5555672L | 250 |
| 5555672L | 500 |
+-----+-----+
```

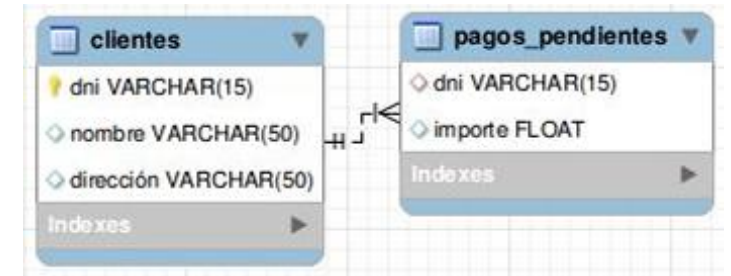
- Realizar los siguientes pasos sobre la base de datos clientes_pagos: (Realizar el diseño físico e insertar los datos como aparece en las capturas)

```
START TRANSACTION;
UPDATE pagos_pendientes
SET importe=300 WHERE importe=250;
INSERT INTO clientes
VALUES ('12345678M', 'Paquirrin', 'La finca de Mamma');
INSERT INTO pagos_pendientes
VALUES ('12345678M', 500);
DELETE FROM pagos_pendientes
WHERE importe=500 AND dni=
(SELECT dni FROM clientes
WHERE nombre='Pepe Cifuentes');
select * from clientes;
select * from pagos_pendientes;
COMMIT;
```

```
mysql> select * from pagos_pendientes;
+-----+-----+
| dni   | importe |
+-----+-----+
| 5555672L | 300 |
| 12345678M | 500 |
+-----+-----+
```

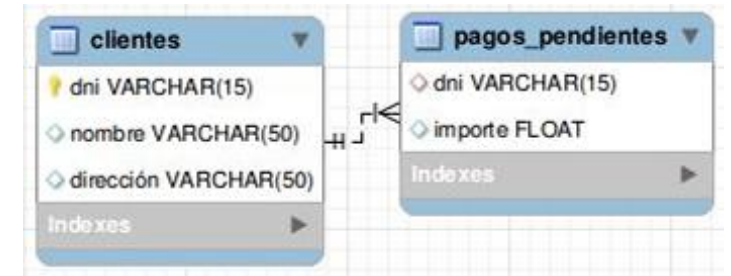
```
mysql> select * from clientes;
+-----+-----+-----+
| dni   | nombre      | dirección      |
+-----+-----+-----+
| 12345678M | Paquirrin    | La finca de Mamma |
| 5555672L | Pepe Cifuentes | C/Los almendros,23 |
+-----+-----+-----+
```

EJERCICIO 18



- Realizar los siguientes pasos sobre la base de datos clientes_pagos:
 1. Desactivar la confirmación automática
 2. Insertar un nuevo cliente con tu nombre dirección y dni
 3. Insertar un pago pendiente de 250€ al nuevo cliente
 4. Añadir un 7% a todos los importes por encima de 10€
 5. Insertar un nuevo pago para el dni 1234 y con el importe máximo del dni 8899
 6. Insertar un nuevo pago para el cliente con "tu nombre" que sea igual a la suma de todos tus pagos pendientes hasta el momento
 7. Verificar los cambios
 8. Confirmar la Transacción
-

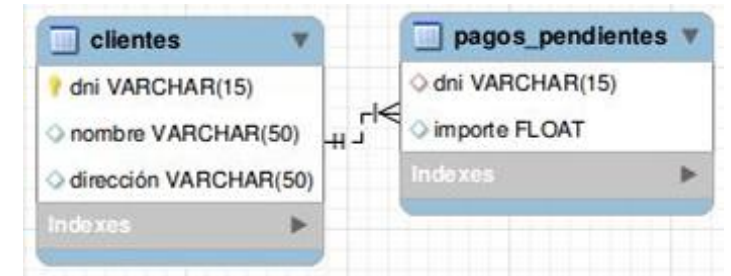
EJERCICIO 18



- Realizar los siguientes pasos sobre la base de datos clientes_pagos:

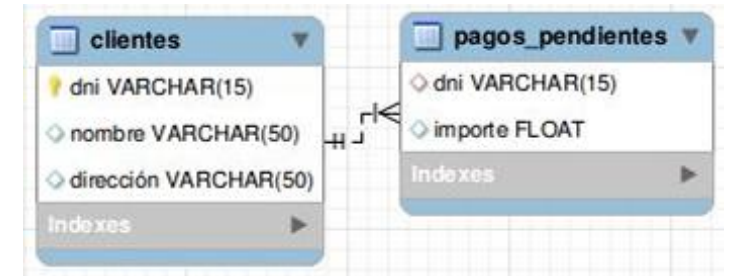
```
SET AUTOCOMMIT=0;
INSERT INTO clientes
VALUES ('12345699M', 'Eduardo', 'La finca de Mamma');
INSERT INTO pagos_pendientes
VALUES ('12345699M', 250);
UPDATE pagos_pendientes
SET importe=importe+importe*7/100
WHERE importe>10;
INSERT INTO pagos_pendientes
SELECT '1234', max(importe)
FROM pagos_pendientes WHERE dni = '8899';
INSERT INTO pagos_pendientes
SELECT
    (SELECT dni FROM clientes WHERE nombre='Eduardo'),
    (SELECT SUM(importe) FROM pagos_pendientes) ;
COMMIT;
```

EJERCICIO 19



- Realizar los siguientes pasos sobre la base de datos `clientes_pagos`, crea una tabla temporal `clientes2`:
 1. Verifica el estado de la confirmación automática; `SHOW VARIABLES WHERE Variable_name='autocommit'`
 2. Modifica tu dirección con la dirección de Cifu
 3. Modifica los máximos importes de cada usuario sumando 10€
 4. Eliminar todos los pagos a "tu nombre" que estén por debajo de tu media
 5. Verificar los cambios
 6. Confirmar la Transacción
-

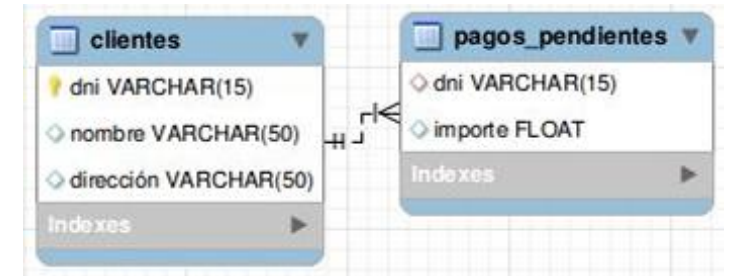
EJERCICIO 19



- Realizar los siguientes pasos sobre la base de datos clientes_pagos, crea una tabla temporal clientes2:

```
UPDATE clientes
SET dirección =
    (SELECT dirección FROM clientes
     WHERE nombre = 'Cifu')
WHERE nombre='Eduardo';
UPDATE pagos_pendientes
SET importe=importe+10
WHERE importe =
    (SELECT max(importe)
     FROM pagos_pendientes2
     group by dni);
DELETE FROM pagos_pendientes
WHERE dni=(SELECT dni FROM clientes WHERE nombre='Eduardo')
AND importe<
    (SELECT avg(importe) FROM pagos_pendientes2
     WHERE dni=
        (SELECT dni FROM clientes
         WHERE nombre='Eduardo'));
```

EJERCICIO 20

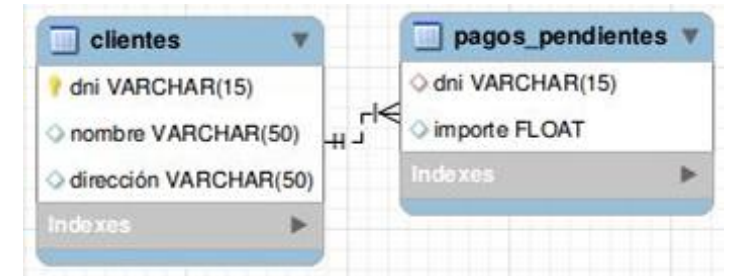


- Modifica la clave foránea dni de la tabla pagos_pendientes para que cuando se elimine un cliente se eliminen todos sus pagos.
- Modifica la clave foránea dni de la tabla pagos_pendientes para que cuando se actualice el dni de un cliente se pongan a nulo todos sus pagos.

```
show create table pagos_pendientes;
```

```
1 CREATE TABLE `pagos_pendientes` (  
2   `dni` varchar(15) DEFAULT NULL,  
3   `importe` float DEFAULT NULL,  
4   KEY `dni` (`dni`),  
5   CONSTRAINT `pagos_pendientes_ibfk_1` FOREIGN KEY (`dni`) REFERENCES  
6   `clientes` (`dni`)  
7 ) ENGINE=InnoDB DEFAULT CHARSET=latin1
```

EJERCICIO 20



- Modifica la clave foránea dni de la tabla pagos_pendientes para que cuando se elimine un cliente se eliminen todos sus pagos.
- Modifica la clave foránea dni de la tabla pagos_pendientes para que cuando se actualice el dni de un cliente se pongan a nulo todos sus pagos.

```
show create table pagos_pendientes;
```

```
1 CREATE TABLE `pagos_pendientes` (  
2   `dni` varchar(15) DEFAULT NULL,  
3   `importe` float DEFAULT NULL,  
4   KEY `dni` (`dni`),  
5   CONSTRAINT `pagos_pendientes_ibfk_1` FOREIGN KEY (`dni`) REFERENCES  
6   `clientes` (`dni`)  
   ) ENGINE=InnoDB DEFAULT CHARSET=latin1
```

```
ALTER TABLE pagos_pendientes DROP  
FOREIGN KEY pagos_pendientes_ibfk_1;  
ALTER TABLE pagos_pendientes ADD  
FOREIGN KEY (dni) REFERENCES clientes(dni)  
ON DELETE CASCADE  
ON UPDATE SET NULL;
```

20. ACCESO CONCURRENTE A LOS DATOS

- Cuando se utilizan transacciones, pueden suceder problemas de concurrencia en el acceso a los datos, es decir, **problemas ocasionados por el acceso al mismo dato de dos transacciones distintas.**

20. ACCESO CONCURRENTES A LOS DATOS

- Estos problemas están descritos por SQL estándar y son los siguientes:
 - **Dirty Read** (Lectura Sucia). Una transacción lee datos escritos por una transacción que no ha hecho COMMIT.
 - **Actualizaciones perdidas**. Dos transacciones acceden a la vez al mismo dato.
 - **Nonrepeatable Read** (Lectura No Repetible). Se produce cuando una transacción consulta el mismo dato dos veces durante la ejecución de la transacción y la segunda vez encuentra que el valor del dato ha sido modificado por otra transacción.
 - **Phantom Read** (Lectura Fantasma). Este error ocurre cuando una transacción ejecuta dos veces una consulta que devuelve un conjunto de filas y en la segunda ejecución de la consulta aparecen nuevas filas en el conjunto que no existían cuando se inició la transacción.
-

20. ACCESO CONCURRENTE A LOS DATOS

- EJEMPLO

- Si tenemos en la tabla RECAMBIOS una columna denominada Stock cuyo valor inicial es 12 unidades, supongamos que dos transacciones concurrentes actualizan el valor del stock de la siguiente forma:

Transacción 1: compra de 100 unidades de ese recambio. Transacción 2: venta de 5 unidades del recambio.
--

20. ACCESO CONCURRENTE A LOS DATOS

- Lectura sucia (Dirty Read)

Transacción 1: compra de 100 unidades de ese recambio. Transacción 2: venta de 5 unidades del recambio.
--

- La ejecución normal de las transacciones debería ser:

TRANSACCION	OPERACIÓN	VALOR Stock
Transacción 1	Leer Stock	12
Transacción 1	Stock=12 + 100	112
Transacción 2	Leer Stock	112
Transacción 2	Stock=112-5	107

- Se produce cuando antes de finalizar una transacción, otra accede a los mismos datos, por tanto, la segunda transacción actualiza el dato inicial y la primera modificación se pierde. ***
-

20. ACCESO CONCURRENTE A LOS DATOS

- Lectura sucia (Dirty Read)

Transacción 1: **compra** de 100 unidades de ese recambio.
Transacción 2: **venta** de 5 unidades del recambio.

- Cuando una transacción se deshace después de que otra haya accedido a los datos:

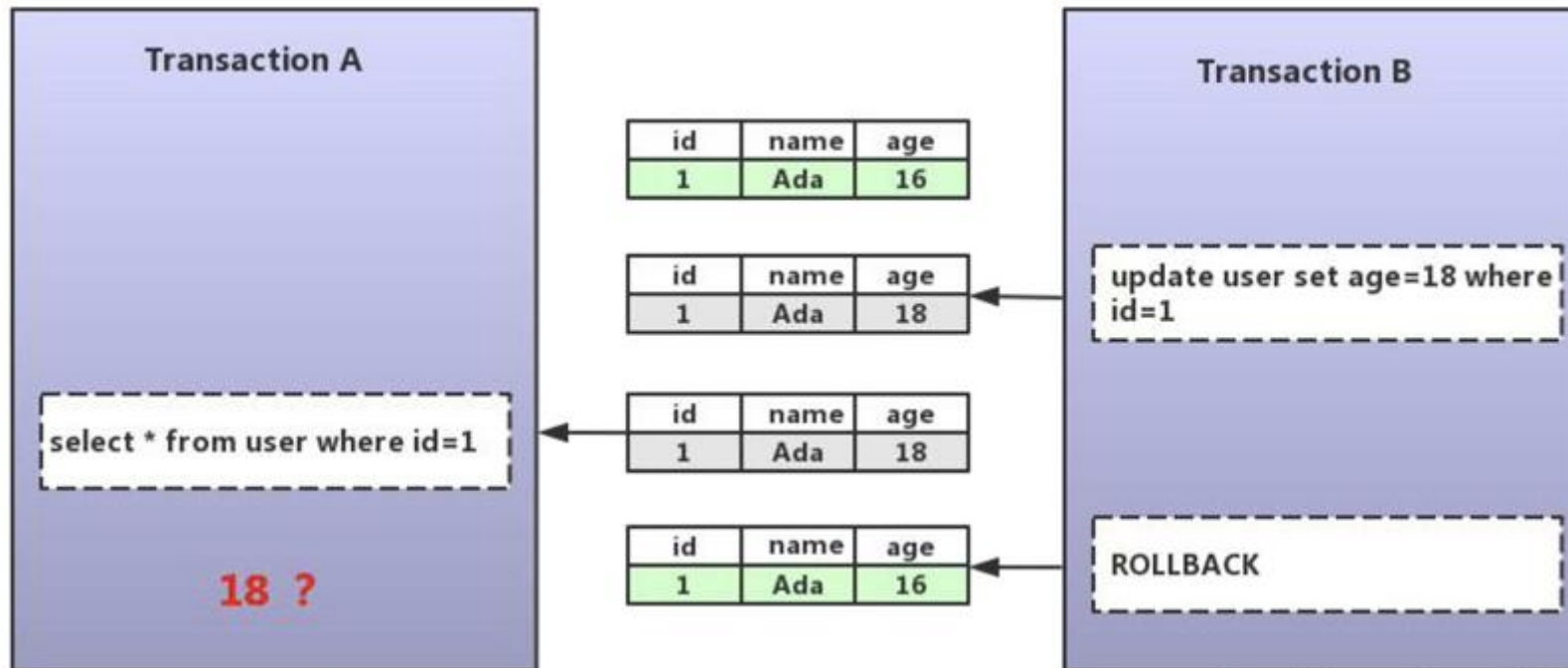
Datos no
comprometidos

TRANSACCIÓN	OPERACIÓN	VALOR Stock
Transacción 1	Leer Stock	12
Transacción 1	Stock =12 +100	112
Transacción 2	Leer Stock	112
Transacción 2	Stock =112 -5	107
Transacción 1	ROLLBACK	12

- Si en este momento la transacción 2 consulta el stock observará que es 12
-

20. ACCESO CONCURRENTES A LOS DATOS

- Lectura sucia (Dirty Read)



20. ACCESO CONCURRENTE A LOS DATOS

- Actualizaciones perdidas

Transacción 1: compra de 100 unidades de ese recambio. Transacción 2: venta de 5 unidades del recambio.
--

- Cuando una transacción accede a un valor antes de que la transacción anterior haya finalizado:

Actualización
perdida

TRANSACCIÓN	OPERACIÓN	VALOR Stock
Transacción 1	Leer Stock	12
Transacción 2	Leer Stock	12
Transacción 1	Stock =12 +100	112
Transacción 2	Stock=12-5	7

20. ACCESO CONCURRENTE A LOS DATOS

- Actualizaciones perdidas
- Otro ejemplo:

Transacción 1	Transacción 2
<code>UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1;</code>	
	<code>SELECT saldo FROM cuentas WHERE id = 1;</code>
<code>ROLLBACK</code>	

20. ACCESO CONCURRENTE A LOS DATOS

- Lectura no repetible (Non Repeatable Read)

Transacción 1: **compra** de 100 unidades de ese recambio.
Transacción 2: **venta** de 5 unidades del recambio.

- Cuando una transacción se deshace después de que otra haya accedido a los datos:

Datos no
comprometidos

TRANSACCIÓN	OPERACIÓN	VALOR Stock
Transacción 1	Leer Stock	12
Transacción 1	Stock =12 +100	112
Transacción 2	Leer Stock	112
Transacción 2	Stock =112 -5	107
Transacción 1	ROLLBACK	12

- Si en este momento la transacción 2 consulta el stock observará que es 12
-

20. ACCESO CONCURRENTE A LOS DATOS

- Lectura no repetible (Non Repeatable Read)
 - Se produce, por ejemplo, cuando una transacción lee datos que ya ha leído y detecta que han cambiado.

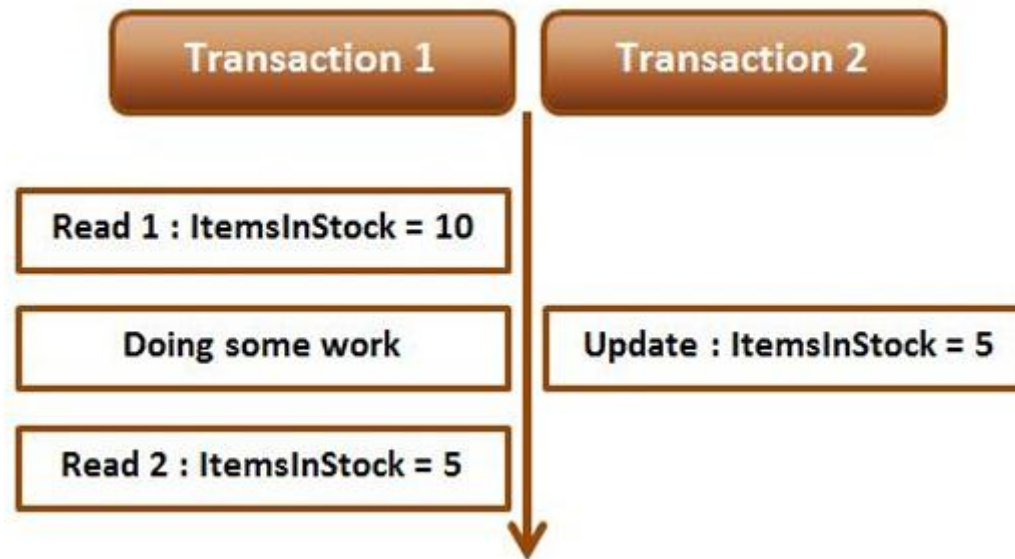
- Ejemplo:

TRANSACCIÓN	OPERACIÓN	VALOR Stock
Transacción 1	Leer Stock	12
Transacción 2	Actualiza stock a 14 y COMMIT	14
Transacción 1	Leer Stock	14

- ¿En qué momento habrá cambiado?
-

20. ACCESO CONCURRENTES A LOS DATOS

- Lectura no repetible (Non Repeatable Read)



20. ACCESO CONCURRENTE A LOS DATOS

- Lectura fantasma (Phantom Read)
 - Ocurre, por ejemplo, cuando una transacción incluye funciones de agrupación (sumar, contar, etc.) sobre unos datos, mientras otra transacción los actualiza.

- EJEMPLO

TRANSACCIÓN	OPERACIÓN	VALOR
Transacción 1	Leer recambios	Se muestran 3 filas
Transacción 2	Insert y COMMIT	Hay 4 recambios (filas)
Transacción 1	Leer recambios	Detecta cambios

20. ACCESO CONCURRENTE A LOS DATOS

- Lectura fantasma (Phantom Read)
 - Si la función lee unos datos antes de actualizar y otros después el resultado es inconsistente.
- EJEMPLO:

Transacción 1: Calcula la cantidad total disponible de Stock.

Transacción 2: Actualiza en 30 unidades la cantidad disponible de Stock de dos de esos productos: Z y V.

20. ACCESO CONCURRENTE A LOS DATOS

- Recuperaciones inconsistentes (Phantom Read)
 - La ejecución normal de estas transacciones es:

IdRecambio	VALOR Stock	VALOR AGREGADO
X	12	12
Y	15	15
Z	20	20+30=50
V	35	35-30=5
W	50	50
U	40	40
TOTAL	172	172

Transacción1

Transacción2

20. ACCESO CONCURRENTE A LOS DATOS

- Recuperaciones inconsistentes (Phantom Read)
 - Cuando la transacción lee unos datos antes de ser modificados y otros después:

TRANSACCIÓN	OPERACIÓN	VALOR Stock	VALOR AGREGADO
Transacción 1	Leer Stock para el IdRecambio X	12	12
Transacción 1	Leer Stock para el IdRecambio Y	15	27
Transacción 2	Leer Stock para el IdRecambio Z	20	
Transacción 2	Modifica el Stock =20+30		
Transacción 1	Leer Stock para el IdRecambio Z	50	77 (X+Y+Z después de modificar)
Transacción 1	Leer Stock para el IdRecambio V	35	112 (X+Y+Z+V antes de modificar)
Transacción 2	Leer Stock para el IdRecambio V	35	
Transacción 2	Modifica el Stock=35-30	5	
Transacción 1	Leer Stock para el IdRecambio W	50	162
Transacción 1	Leer Stock para el IdRecambio U	40	202

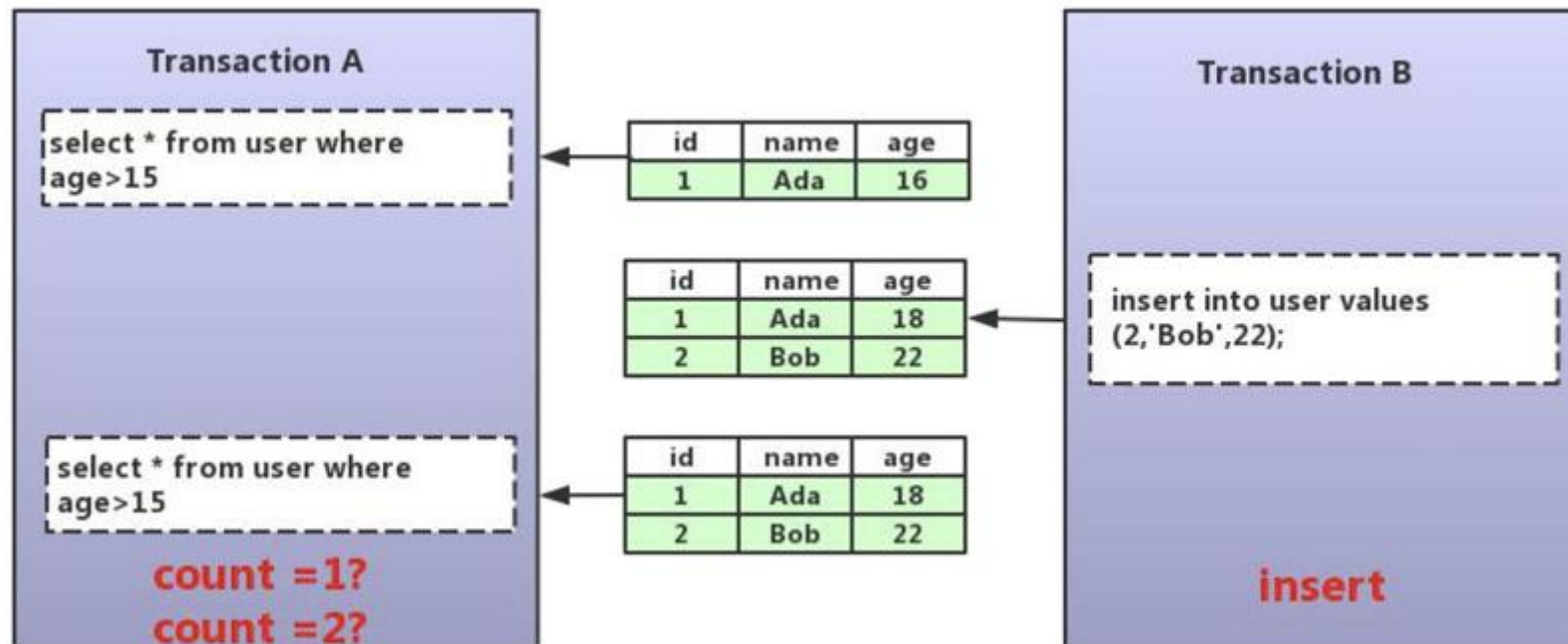
Lectura posterior a la modificación

Lectura anterior a la modificación

- Como vemos, el resultado final no es correcto porque el valor agregado excede en 30 unidades el resultado calculado anteriormente.
-

20. ACCESO CONCURRENTES A LOS DATOS

- Recuperaciones inconsistentes (Phantom Read)



21. NIVEL DE AISLAMIENTO

- Cuando se trabaja con transacciones, el SGBD puede bloquear conjuntos de datos para evitar o permitir que sucedan estos problemas.
 - Según el nivel de concurrencia que se desee, es posible solicitar al SGBD **cuatro niveles de aislamiento**.
 - **Un nivel de aislamiento define cómo los cambios hechos por una transacción son visibles a otras transacciones.**
-

21. NIVEL DE AISLAMIENTO

- **Read Uncommitted (Lectura no acometida)**
 - Permite que sucedan los tres problemas (dirty, phantom y non repeatable).
 - **Las sentencias SELECT son efectuadas sin realizar bloqueos**, por tanto, todos los cambios hechos por una transacción pueden verlos las otras transacciones.
 - **Read Committed (Lectura acometida)**
 - Se pueden dar los problemas de Phantom Read y Non Repeteable Read.
 - **Dentro de una transacción únicamente se pueden ver los cambios de las transacciones ya comprometidas.**
-

21. NIVEL DE AISLAMIENTO

- **Repeatable Read (Lectura Repetible)**

- Tan solo se permite el problema del Phantom Read.
- Consiste en que **ningún registro leído con un SELECT se puede cambiar** en otra transacción.

- **Serializable**

- Soluciona todos los problemas de concurrencia
 - Las transacciones ocurren de forma totalmente aislada a otras transacciones.
 - Se bloquean las transacciones de tal manera que ocurren unas detrás de otras, sin capacidad de concurrencia.
 - El SGBD las ejecuta concurrentemente si puede asegurar que no hay conflicto con el acceso a los datos.
-

21. NIVEL DE AISLAMIENTO

Nivel de aislamiento	Lecturas fantasma	Lecturas no repetibles	Lecturas sucias
Serializable	No	No	No
Repeatable Read	Si	No	No
Read Committed	Si	Si	No
Read Uncommitted	Si	Si	Si

21. NIVEL DE AISLAMIENTO

- ¿Cuál es el nivel de aislamiento por defecto?

```
SHOW VARIABLES LIKE '%isolation%';  
select @@tx_isolation;
```

tx_isolation

Property	Value
Deprecated	5.7.20
System Variable	tx_isolation
Scope	Global, Session
Dynamic	Yes
Type	enumeration
Default Value	REPEATABLE-READ
Valid Values	READ-UNCOMMITTED READ-COMMITTED REPEATABLE-READ SERIALIZABLE

21. NIVEL DE AISLAMIENTO

- Para cambiar el nivel de aislamiento:

```
1  SET [GLOBAL | SESSION] TRANSACTION ISOLATION LEVEL
2  {
3      REPEATABLE READ
4      | READ COMMITTED
5      | READ UNCOMMITTED
6      | SERIALIZABLE
7  }
```

21. NIVEL DE AISLAMIENTO

- Puedes establecer el nivel de aislamiento globalmente, para la sesión actual o para la siguiente transacción:
 - **Con GLOBAL**, establece un nivel de aislamiento global para todas las sesiones siguientes. Las sesiones existentes no se ven afectadas.
 - **SESSION**, establece un nivel de aislamiento para todas las transacciones posteriores a las realizadas dentro de la sesión actual.
 - **Sin ninguna palabra clave (GLOBAL O SESSION)**, la declaración establece un nivel de aislamiento para la siguiente transacción (no iniciada) dentro de la sesión actual. Tras ella, el resto de transacciones vuelven a utilizar el nivel de aislamiento actual (SESSION).
-

21. NIVEL DE AISLAMIENTO

- Establecer nivel de aislamiento de transacción sin GLOBAL o SESSION no está permitido mientras hay una transacción activa:

```
1  mysql> START TRANSACTION;  
2  Query OK, 0 rows affected (0.02 sec)  
3  
4  mysql> SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;  
5  ERROR 1568 (25001): Transaction isolation level can't be changed  
6  while a transaction is in progress
```

22. MÉTODOS DE BLOQUEO

- Como estás viendo, estos problemas se producen porque **las operaciones en una transacción se ejecutan en serie** y éstas no se pueden hacer en cualquier orden cuando operan sobre los mismos datos o sobre datos relacionados.
 - El propio SGBD mediante el planificador se encarga de establecer el orden en que se deben **realizar las operaciones** incluidas en las transacciones concurrentes garantizando además el uso eficiente del procesador.
-

22. MÉTODOS DE BLOQUEO

- El planificador establece este orden basándose en algoritmos de control de concurrencia, que dan lugar a **distintos métodos de bloqueo**.
 - Veremos a continuación cuáles son esos métodos.
-

23. POLÍTICAS DE BLOQUEO

- El **bloqueo** consiste en que el uso de los datos de una transacción es **exclusivo**, es decir que otra transacción concurrente no puede tener acceso a los datos que están siendo usados por la anterior.
 - El bloqueo se produce antes de iniciar la transacción y se libera cuando la transacción se completa.
 - Este bloqueo se puede producir a nivel de:
 - Base de datos.
 - Tabla.
 - Fila.
 - Atributo.
-

23. POLÍTICAS DE BLOQUEO

- Algunos SGBD, como MySQL, permiten elegir el nivel de bloqueo, aunque hay sentencias como **ALTER TABLE** que necesariamente producen bloqueo a nivel de tabla.
 - En el caso de las tablas **InnoDB** de MySQL por defecto el bloqueo se produce a nivel de fila.
 - En las tablas **MyISAM** es a nivel de tabla.
-

23. POLÍTICAS DE BLOQUEO

- Independientemente del nivel de bloqueo un **SGBD** puede utilizar diferentes tipos de bloqueo:
 - Bloqueo binario
 - Bloqueo compartido/exclusivo
 - Bloqueo exclusivo
 - Bloqueo compartido
-

23. POLÍTICAS DE BLOQUEO

- Bloqueo binario
 - Es la forma más simple de bloquear, ya que en este método cada transacción debe solicitar:
 - El bloqueo del elemento que vaya a utilizar antes de acceder a él mediante una operación de bloquear
 - Liberar todos los bloqueos mediante una operación de desbloquear para que otra transacción los pueda utilizar.
 - Se denomina bloqueo binario porque sólo tiene dos estados, bloqueado (1) o desbloqueado (0).
-

23. POLÍTICAS DE BLOQUEO

- Bloqueo binario
 - La implementación de este método es muy simple, basta con mantener una tabla que indique qué partes de los datos están bloqueados y por qué transacción, además estas operaciones son programadas y manejadas por el SGBD y el usuario no tiene que preocuparse.
 - Cada SGBD tiene un mecanismo de bloqueo predeterminado.
 - **Los bloqueos binarios son demasiado restrictivos porque bloquean incluso las operaciones que no son conflictivas porque no actualizan datos (como un select).**
-

23. POLÍTICAS DE BLOQUEO

- Bloqueo compartido/exclusivo
 - Este método es menos restrictivo porque permite la compatibilidad de algunos bloqueos.
 - **Bloqueo exclusivo:**
 - Cuando el objeto sólo es accesible por la transacción que lo bloqueó.
 - Se utiliza cuando al menos una de las operaciones de la transacción es de escritura.
 - **Bloqueo compartido:**
 - Cuando todas las transacciones concurrentes son sólo de lectura.
 - Los bloqueos compartidos permiten que varias transacciones lean a la vez el mismo elemento de datos.
-

23. POLÍTICAS DE BLOQUEO

- Bloqueo compartido/exclusivo
 - Este tipo de bloqueo puede tener 3 estados:
 - Desbloqueado
 - Compartido
 - Exclusivo.
-

23. POLÍTICAS DE BLOQUEO

- Bloqueo compartido/exclusivo
 - **La posibilidad de compartir el bloqueo hace que el acceso sea más eficiente, pero este sistema aumenta la carga de trabajo del sistema.**
 - Aunque los bloqueos impiden las inconsistencias serias de los datos, su uso puede conducir problemas importantes.

23. POLÍTICAS DE BLOQUEO

- Problemas de bloqueo
 - Uno de estos problemas es que algunas transacciones pueden quedar interbloqueadas dependiendo del nivel de aislamiento seleccionado.
 - Esta situación de **interbloqueo**, o **deadlock** entre dos transacciones, consiste en que ninguna de ellas puede seguir ejecutándose porque la primera intenta acceder a un bloque de datos que tiene bloqueada la segunda, y la segunda, intenta acceder a un bloque de datos que tiene bloqueada la primera.
-

23. POLÍTICAS DE BLOQUEO

- Problemas de bloqueo
 - **En esta situación de interbloqueo, el SGBD elimina a una de las dos transacciones, siendo la víctima del interbloqueo la que hace rollback de sus operaciones.**
 - Intenta escoger para cancelar transacciones pequeñas, el tamaño de la transacción es determinado por el número de filas insertadas, actualizadas, o eliminadas.

24. BLOQUEO DE DATOS

- Normalmente el administrador del bloqueo del servidor bloquea las tablas o filas automáticamente.
- En MySQL el bloqueo de tablas se puede hacer de forma explícita mediante el uso de dos comandos:
 - LOCK TABLES / UNLOCK TABLES

24. BLOQUEO DE DATOS

- La sintaxis de estos comandos es la siguiente:

```
1  LOCK TABLES
2      tbl_name [[AS] alias] lock_type
3      [, tbl_name [[AS] alias] lock_type] ...
4
5  lock_type:
6      READ [LOCAL]
7      | [LOW_PRIORITY] WRITE
8
9  UNLOCK TABLES
```

- Una sesión no puede acceder a bloqueos de otra sesión ni puede desbloquearlos.
-

24. BLOQUEO DE DATOS

- Clave Foránea
 - Al bloquear una tabla con LOCK TABLES cualquier tabla relacionada con una clave foránea queda abierta y bloqueada implícitamente.

```
1  mysql> LOCK TABLES t1 READ;
2  mysql> SELECT COUNT(*) FROM t1;
3  +-----+
4  | COUNT(*) |
5  +-----+
6  |        3 |
7  +-----+
8  mysql> SELECT COUNT(*) FROM t2;
9  ERROR 1100 (HY000): Table 't2' was not locked with LOCK TABLES
```

25. RESUMEN

